

Rapport de Projet

Session 2024/2025

Un projet d'artiste



Réalisé par Cathy NAVARRO

Pour le compte de Nana-Sensei

Sommaire

I.	Introduction.....	1
II.	Le projet.....	1
A.	Présentation.....	1
B.	Conception	2
1.	<i>Gestion du projet</i>	2
2.	<i>Persona et user story</i>	3
3.	<i>Design</i>	3
4.	<i>Base de données</i>	5
C.	Développement	8
1.	<i>Choix techniques</i>	8
2.	<i>Réalisations</i>	9
3.	<i>Le déploiement</i>	61
4.	<i>Les limites et voies d'amélioration du projet</i>	62
III.	Conclusion.....	63

I. Introduction

L'utilisation d'internet pour se construire une identité quand on est artiste est, de nos jours, inévitable. Les compétences à maîtriser pour être présent de manière efficaces, en limitant les pertes de public sont nombreuses. C'est pourquoi le commanditaire du projet détaillé plus loin dans ce rapport a fait appel à mes compétences en développement web ainsi qu'aux compétences transverses acquises durant mes différentes expériences professionnelles.

II. Le projet

A. Présentation

Ce projet repose sur un besoin de clarté pour l'artiste comme pour sa communauté sur ses activités et leur gestion.

Lors d'un échange avec elle sur sa pratique, elle m'explique :

- Donner des cours de dessins en groupe ou individuel dans les alentours de sa commune,
- Participer à des conventions durant lesquelles elle vend ses créations et fanarts,
- Alimenter une boutique en ligne qui lui permet d'apposer ses designs sur tout type de supports,
- Partager des conseils et faire grandir sa communauté sur les réseaux sociaux.

Ces activités apportent leur lot de problématiques :

- Il n'est pas évident de prendre rendez-vous pour des cours individuels et il faut souvent se déplacer pour les cours collectifs,
- Les invendus des conventions sont stockés chez l'artiste entre les conventions et n'ont pas la possibilité d'être vendu autrement,
- La plateforme qui vend ses designs prend une commission non négligeable sur les ventes et ne fait aucune remise lors de commande de ses propres designs,
- Le contenu qui peut être posté est limité sur les réseaux sociaux (durée des vidéos, longueur du texte, interactions avec la communauté...).

Pour répondre à toutes ces problématiques, je lui ai proposé la création d'un site internet qui regroupera toutes ses activités avec des visuels propres à son univers.

Le site se sépare donc en trois parties :

- Les cours : actuellement une simple page explicative, un agenda sera ajouté lorsque l'artiste aura possibilité de proposer des créneaux de formations,
- La boutique en ligne : afin de vendre les invendus des conventions ainsi que du contenu inédit,
- Un blog où elle pourra partager du contenu plus variés que sur les réseaux sociaux et échanger via les commentaires avec la communauté.

2. Persona et user story

Des recherches sur ses différents réseaux sociaux, l'étude de la concurrence ainsi que l'échange avec l'artiste m'ont aidé à mieux comprendre l'univers de l'artiste et le public visé par ce site internet et donc adapter mes choix lors des propositions de fonctionnalités.

J'ai pu extraire 2 types de personnes qui pourraient utiliser son site internet et les moyens de faire connaître son site et de fidéliser les utilisateurs.

a) Julien

Il a entre 16 et 25 ans, étudiant, fan de manga, fait du dessin en amateur depuis plusieurs années. Il est à la recherche d'astuces et tutos pour s'améliorer, il suit donc le compte de l'artiste sur Instagram où il aime les posts et parfois laisser un commentaire. Il fait des achats sur internet.

Il découvre l'artiste en convention autour des mangas, il visitera le site pour voir les tutos dessin qui s'y trouvent. Il pourra interagir sur les articles du blog après s'être inscrit. Il visitera la boutique pour pouvoir acheter ce qu'il n'a pas pu en convention.

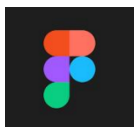
b) Elodie

Elle a environ 30 ans, assistante de direction, mère de 2 enfants de 7 et 10ans. Celui de 7 ans veut apprendre à dessiner et elle cherche un professeur de confiance qui fera progresser son enfant dans une ambiance agréable. Elle a l'habitude d'utiliser Vinted pour refaire la garde-robe de ses enfants régulièrement.

Elle est à la recherche de cours de dessins en ligne pour son enfant. Après consultation des retours des anciens élèves et des parents d'anciens élève, elle réserve un créneau de cours en ligne. Après une première expérience concluante, elle prend un accompagnement plus complet et en parle avec ses ami(e)s et/ou collègues.

Grâce à ces user stories, j'ai pu me faire une idée du chemin le plus rapide pour atteindre le but de chacun, maximisant leur adhésion et leurs actions sur le site.

3. Design

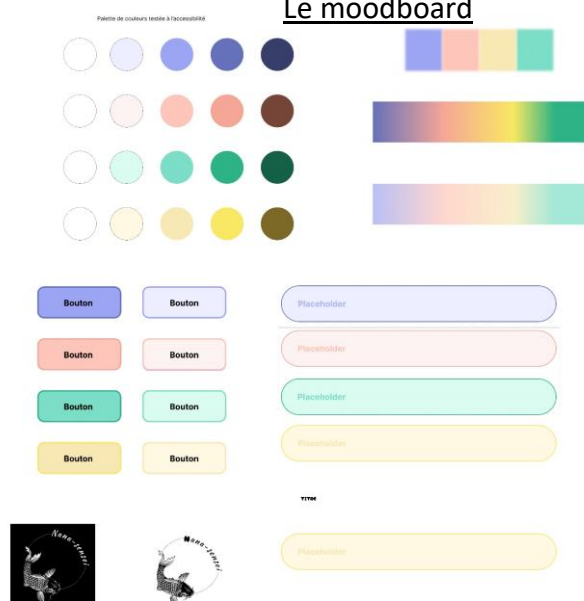


Le design a été travaillé à partir du logiciel **Figma**. Pour proposer un design cohérent, j'ai dans un premier temps demandé si l'artiste avait déjà ou non une charte graphique ou non, et s'il y avait des tons ou des éléments qu'il souhaitait en particulier.

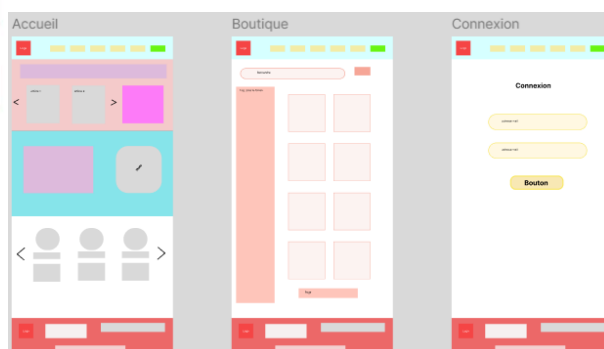
De ces informations, j'ai donc créé un moodboard (planche d'ambiance qui sert à définir l'orientation visuelle d'un projet créatif), comprenant la palette de couleur, le logo ainsi que certains composants visuels. Pour m'assurer qu'ils seront utilisables sur le site que j'allais développer, il m'a semblé important de vérifier leur accessibilité en terme de contraste pour les couleurs, d'espacement et de taille pour ce qui concerne les boutons et enfin de lisibilité pour ce qui est des polices d'écriture. Une pédagogie autour de l'écoconception

et de l'accessibilité ont permis plusieurs ajustements et une nouvelle réflexion sur les fonctionnalités à apporter sur le site présenté grâce à un wireframe (outil de planification visuelle structurant le contenu et les fonctionnalités d'un site web clairement).

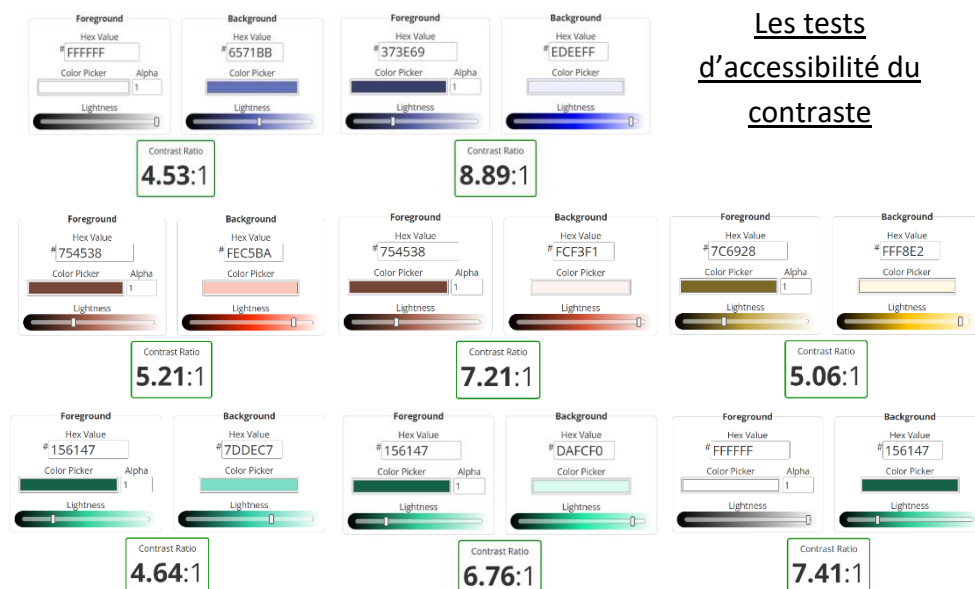
Le moodboard



Le wireframe des 3 pages principales

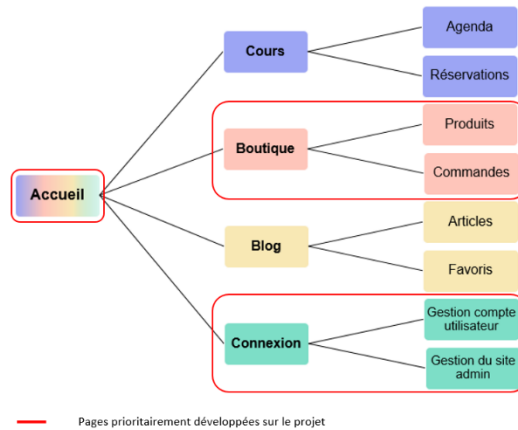


Les tests d'accessibilité du contraste



J'ai réalisé la maquette en fonction de tous les éléments collectés précédemment afin d'obtenir un site internet qui respecte à la fois les contraintes liées à l'univers visuels de l'artiste et celles liées à l'accessibilité. Je me suis concentrée sur la boutique puisque les deux thématiques du site sont en plein remaniement et les précisions de fonctionnement n'arriveront que milieu 2025. Avec l'essor de la consultation des sites internet sur smartphone, il a été choisi de produire une mise en page pour les ordinateurs et les tablettes en paysage et une autre pour les smartphones et les tablettes en portrait pour que la navigation soit agréable peu importe le média utilisé pour le consulter.

Pour plus de clarté sur les liens entre les pages, un schéma indiquant les issus de chaque thématique a été produit et a servi de repère dans le développement des fonctionnalités afin d'identifier celles prioritaires.

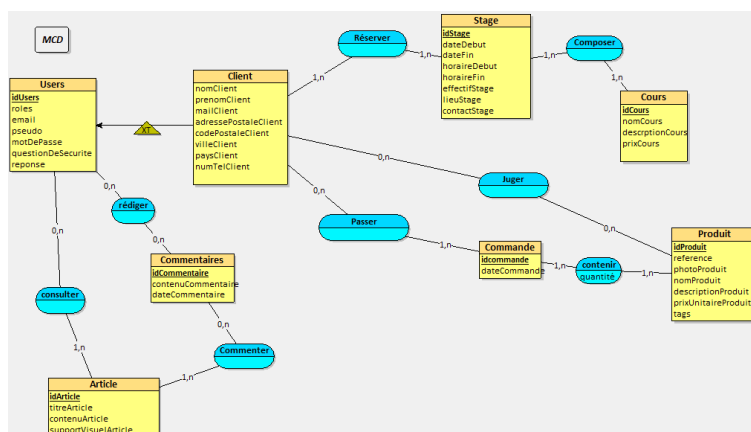


4. Base de données

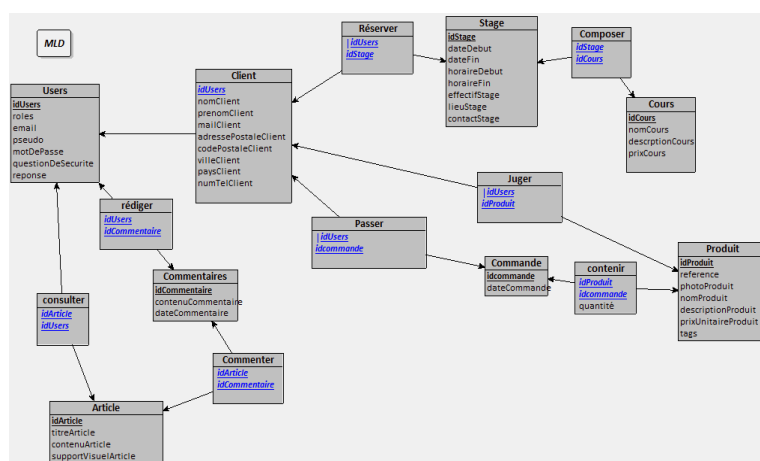
	Nom	Format	longueur	type	
				Élémentaire	calculée
client	nomClient	alphabétique	50	x	
	prenomClient	alphabétique	50	x	
	mailClient	alphanumérique	100	x	
	adressePostaleClient	alphanumérique	150	x	
	codePostaleClient	alphanumérique	100	x	
	villeClient	alphanumérique	100	x	
	paysClient	alphanumérique	50	x	
	numtelClient	alphabétique	15	x	
users	idUser	alphanumérique		x	
	roles	alphanumérique	40	x	
	email	alphanumérique	100	x	
	Pseudo	alphanumérique	50	x	
	motDePasse	alphanumérique	150	x	
	questionDeSecurite	alphanumérique	100	x	
	reponse	alphanumérique	100	x	
produit	idProduit	alphanumérique		x	
	reference	alphanumérique	45	x	
	photoProduit	alphanumérique	1000	x	
	nomProduit	alphanumérique	50	x	
	descriptionProduit	alphanumérique	300	x	
	prixUnitaireProduit	numérique		x	
	tags	alphabétique	45	x	
commande	idCommande	alphanumérique		x	
	dateCommande	numérique		x	
	quantiteCommande	numérique		x	
article	idArticle	alphanumérique		x	
	titreArticle	alphanumérique	100	x	
	contenuArticle	alphanumérique	2000	x	
	supportVisuelArticle	alphanumérique	1000	x	
commentaire	idCommentaire	alphanumérique		x	
	contenuCommentaire	alphanumérique		x	
cours	dateCommentaire	numérique		x	
	idCours	alphanumérique		x	
	nomCours	alphanumérique		x	
	descriptionCours	alphanumérique		x	
	prixCours	numérique		x	
stage	idStage	numérique		x	
	dateDebut	date		x	
	dateFin	date		x	
	horaireDebut	date		x	
	horaireFin	date		x	
	effectifStage	numérique		x	
	lieuStage	alphanumérique	100	x	
	contactStage	alphanumérique	100	x	

Afin de pouvoir développer un site internet fonctionnel, il est important d'avoir une base de données réfléchie. A l'aide du dossier de conception et de la méthode MERISE, j'ai donc mis en place un dictionnaire de données, me permettant de prendre en compte et de lister toutes les informations que devra traiter par la suite ma base de données.

Une fois obtenu et validé, j'utilise ce dictionnaire de données pour créer mon Modèle Conceptuel de Donnée (MCD) grâce auquel j'établis les liens entre les différentes entités. Il va décrire de façon formelle les données utilisées par le système d'information et sa représentation graphique, simple et accessible permettra à mon commanditaire de participer à son élaboration.



L'étape suivante est le Modèle Logique de Donnée (MLD). Il permet de lier les entités non plus par des relations graphiques mais par les clés étrangères déterminées par des règles spécifiques par rapport à la cardinalité. Il sert de passerelle entre le MCD qui est très visuel de par sa représentation graphique et le Modèle Physique de Données (MPD) qui est la dernière étape avant la rédaction du script pour la création de la base de données et de ses tables.



```
Stage = (idStage INT, dateDebut DATE, dateFin DATE, horaireDebut TIME, horaireFin TIME, effectifStage INT, lieuStage VARCHAR(100), contactStage VARCHAR(100));
Produit = (idProduit VARCHAR(50), reference VARCHAR(45), photoProduit VARCHAR(1000), nomProduit VARCHAR(50), descriptionProduit VARCHAR(300), prixUnitaireProduit CURRENCY, tags VARCHAR(45));
Cours = (idCours VARCHAR(50), nomCours VARCHAR(50), descriptionCours VARCHAR(300), prixCours CURRENCY);
Article = (idArticle VARCHAR(50), titreArticle VARCHAR(100), contenuArticle VARCHAR(2000), supportVisuelArticle VARCHAR(100));
Commande = (idcommande INT, dateCommande DATE);
Users = (idUsers VARCHAR(50), roles VARCHAR(40), email VARCHAR(100), pseudo VARCHAR(50), motDePasse VARCHAR(150), questionDeSecurite VARCHAR(100), reponse VARCHAR(100));
Commentaires = (idCommentaire VARCHAR(50), contenuCommentaire VARCHAR(200), dateCommentaire DATE);
Client = (#idUsers, nomClient VARCHAR(50), prenomClient VARCHAR(50), mailClient VARCHAR(100), adressePostaleClient VARCHAR(100), codePostaleClient VARCHAR(100), villeClient VARCHAR(100), paysClient VARCHAR(50), numTelClient VARCHAR(15));
Réserver = (#idUsers, #idStage);
Composer = (#idStage, #idCours);
contenir = (#idProduit, #idcommande, quantité INT);
Passer = (#idUsers, #idcommande);
consulter = (#idArticle, #idUsers);
Juger = (#idUsers, #idProduit);
rédiger = (#idUsers, #idCommentaire);
Commenter = (#idArticle, #idCommentaire);
```

MPD

La dernière étape est la création de la base de données grâce à un script SQL dont l'écriture des lignes est simplifiée grâce aux étapes précédentes. Voici le script pour la création spécifique de la base de donnée de ce projet

```

CREATE TABLE Stage(
    idStage INT,
    dateDebut DATE,
    dateFin DATE,
    horaireDebut TIME,
    horaireFin TIME,
    effectifStage INT,
    lieuStage VARCHAR(100),
    contactStage VARCHAR(100),
    PRIMARY KEY(idStage)
);

CREATE TABLE Produit(
    idProduit VARCHAR(50),
    reference VARCHAR(45),
    photoProduit VARCHAR(1000),
    nomProduit VARCHAR(50),
    descriptionProduit VARCHAR(300),
    prixUnitaireProduit FLOAT,
    tags VARCHAR(45),
    PRIMARY KEY(idProduit)
);

CREATE TABLE Cours(
    idCours VARCHAR(50),
    nomCours VARCHAR(50),
    descriptionCours VARCHAR(300),
    prixCours FLOAT,
    PRIMARY KEY(idCours)
);

CREATE TABLE Article(
    idArticle VARCHAR(50),
    titreArticle VARCHAR(100),
    contenuArticle VARCHAR(2000),
    supportVisuelArticle VARCHAR(1000),
    PRIMARY KEY(idArticle)
);

CREATE TABLE Commande(
    idCommande INT,
    dateCommande DATE,
    PRIMARY KEY(idCommande)
);

CREATE TABLE Users(
    idUsers VARCHAR(50),
    roles VARCHAR(40),
    email VARCHAR(100),
    pseudo VARCHAR(50),
    motDePasse VARCHAR(150),
    questionDeSecurite VARCHAR(100),
    reponse VARCHAR(100),
    PRIMARY KEY(idUsers)
);

CREATE TABLE Commentaires(
    idCommentaire VARCHAR(50),
    contenuCommentaire VARCHAR(200),
    dateCommentaire DATE,
    PRIMARY KEY(idCommentaire)
);

CREATE TABLE Client(
    idUsers VARCHAR(50),
    nomClient VARCHAR(50),
    prenomClient VARCHAR(50),
    mailClient VARCHAR(100),
    adressePostaleClient VARCHAR(150),
    codePostalClient VARCHAR(100),
    villeClient VARCHAR(100),
    paysClient VARCHAR(50),
    numTelClient VARCHAR(15),
    PRIMARY KEY(idUsers),
    FOREIGN KEY(idUsers) REFERENCES Users(idUsers)
);

CREATE TABLE Reserver(
    idUsers VARCHAR(50),
    idStage INT,
    PRIMARY KEY(idUsers, idStage),
    FOREIGN KEY(idUsers) REFERENCES Users(idUsers),
    FOREIGN KEY(idStage) REFERENCES Stage(idStage)
);

CREATE TABLE Composer(
    idStage INT,
    idCours VARCHAR(50),
    PRIMARY KEY(idStage, idCours),
    FOREIGN KEY(idStage) REFERENCES Stage(idStage),
    FOREIGN KEY(idCours) REFERENCES Cours(idCours)
);

CREATE TABLE Contenir(
    idProduit VARCHAR(50),
    idCommande INT,
    quantite INT,
    PRIMARY KEY(idProduit, idCommande),
    FOREIGN KEY(idProduit) REFERENCES Produit(idProduit),
    FOREIGN KEY(idCommande) REFERENCES Commande(idCommande)
);

CREATE TABLE Passer(
    idUsers VARCHAR(50),
    idCommande INT,
    PRIMARY KEY(idUsers, idCommande),
    FOREIGN KEY(idUsers) REFERENCES Users(idUsers),
    FOREIGN KEY(idCommande) REFERENCES Commande(idCommande)
);

CREATE TABLE Consulter(
    idArticle VARCHAR(50),
    idUsers VARCHAR(50),
    PRIMARY KEY(idArticle, idUsers),
    FOREIGN KEY(idArticle) REFERENCES Article(idArticle),
    FOREIGN KEY(idUsers) REFERENCES Users(idUsers)
);

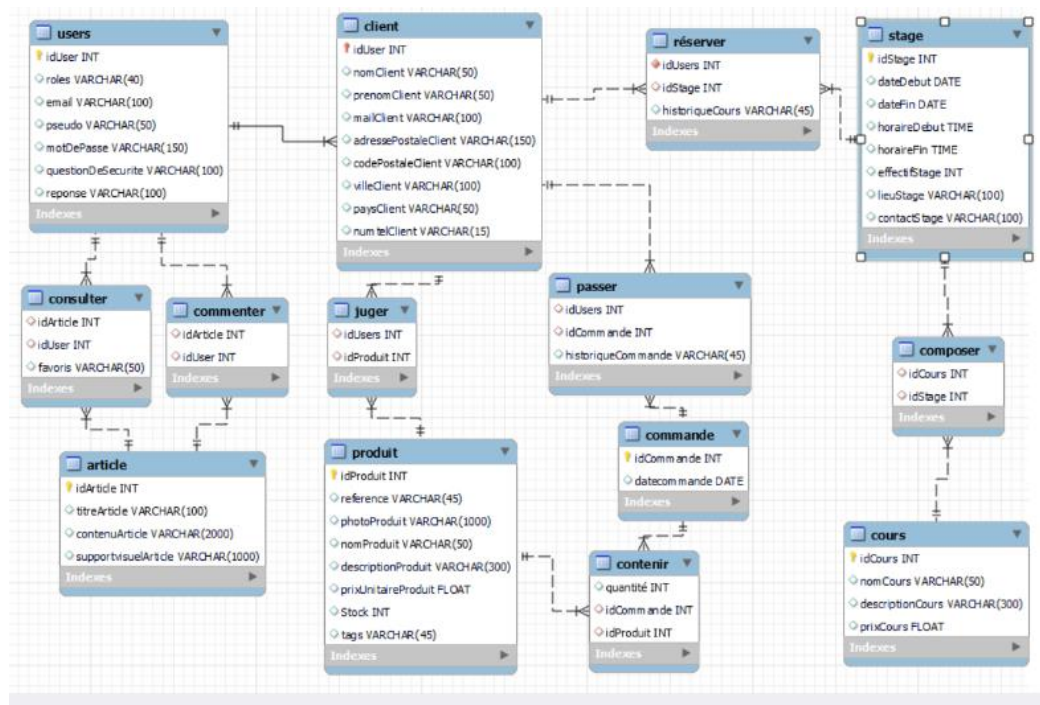
CREATE TABLE Juger(
    idUsers VARCHAR(50),
    idProduit VARCHAR(50),
    PRIMARY KEY(idUsers, idProduit),
    FOREIGN KEY(idUsers) REFERENCES Users(idUsers),
    FOREIGN KEY(idProduit) REFERENCES Produit(idProduit)
);

CREATE TABLE Rédiger(
    idUsers VARCHAR(50),
    idCommentaire VARCHAR(50),
    PRIMARY KEY(idUsers, idCommentaire),
    FOREIGN KEY(idUsers) REFERENCES Users(idUsers),
    FOREIGN KEY(idCommentaire) REFERENCES Commentaires(idCommentaire)
);

CREATE TABLE Commenter(
    idArticle VARCHAR(50),
    idCommentaire VARCHAR(50),
    PRIMARY KEY(idArticle, idCommentaire),
    FOREIGN KEY(idArticle) REFERENCES Article(idArticle),
    FOREIGN KEY(idCommentaire) REFERENCES Commentaires(idCommentaire)
);

```

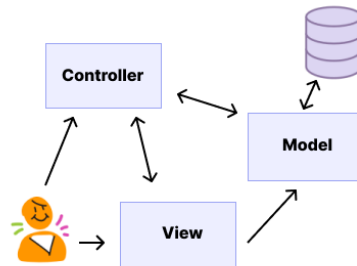
Une fois créée, le reverse Engineering permet d'obtenir ce schéma.



C. Développement

1. Choix techniques

Pour ce projet, j'utilise le MVC (Modèle-Vue-Contrôleur) qui est un modèle d'architecture logicielle qui divise une application en trois composants principaux : le Modèle gère les données et la logique métier, la Vue s'occupe de l'interface utilisateur et de l'affichage, et le Contrôleur agit comme intermédiaire en traitant les entrées de l'utilisateur, en mettant à jour le Modèle et en déterminant quelle Vue afficher.



Pour le mettre en application, j'ai utilisé plusieurs langages afin de répondre au mieux aux problématiques d'accessibilité et de sécurité.



HTML5 : pour le développement des pages statiques du site.



CSS3 : pour ajouter du style aux pages internet.



JavaScript : pour dynamiser mes pages statiques et apporter de nouvelles fonctionnalités.



SQL : pour gérer la base de donnée (création, lecture, modification et suppression).



Java : pour gérer et sécuriser les interactions avec la base de données. L'outil de gestion et d'automatisation de production des projets logiciels utilisé sur ce projet est **Apache Maven** (aussi appelé Maven).

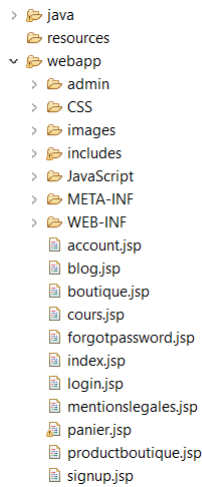


GitHub et Docker : Ce sont deux outils complémentaires. Le premier utilisé pour le versionning et le deuxième pour la création des environnements de développement isolés et reproductibles via des conteneurs ainsi que la facilitation du déploiement



2. Réalisations

Pour une meilleure organisation et pouvoir se retrouver dans les fichiers, ils sont organisés dans des dossiers.



- Le dossier « java » contient tous les packages et toutes les classes du projet.
- Le dossier « admin » contient les pages liés à la partie administrateur.
- Le dossier « CSS » contient toutes les feuilles de style du site.
- Le dossier « images » contient les images et illustrations du site.
- Le dossier « includes » contient les modules communs sur le site (head, menu, footer).
- Le dossier « JavaScript » contient les scripts Javascript utiliser pour dynamiser le site.
- Les pages du site sont hors dossiers pour une facilité de lien entre elles.

a) Les includes

Sur la totalité du site, le head, le menu et le footer sont les mêmes. Ils ont donc été codé dans des fichier à part afin de simplifier sa modification en cas de besoin. Ils sont inclus dans les pages JSP grâce à la commande Java `<%@include file= « includes/head.jsp »%>`, `<%@include file= « includes/menu.jsp »%>`, et `<%@include file= « includes/footer.jsp »%>`.

```
head.jsp

<%@page import="com.nanasenseimvc.model.User"%>
<%@page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Nana-sensei - Accueil</title>

    <style type="text/css">
    <%@include file="../../CSS/header-footer.css" %>
    <%@include file="../../CSS/reset.css" %>
    </style>
    <script src="https://kit.fontawesome.com/1ae84cb769.js" ></script>

    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" >
    <link href="https://fonts.googleapis.com/css2?family=Cherry+Bomb+One&display=swap" rel="stylesheet">
</head>
```

Le « head.jsp » me permet de regrouper tous les liens et informations communs sur toutes les pages comme les polices, le CSS de reset pour que les navigateurs n'interfèrent pas avec le style que j'ai codé, le CSS du header et du footer qui sont communs à toutes les pages.

Le menu est également présent sur toutes les pages. Celui respect au mieux les règles d'accessibilité.

Celui-ci contient un menu classique utiliser dans la version ordinateur et tablette et un menu burger utiliser pour la version mobile. Son attribution est gérée via des média queries.

La récupération de l'information de connexion permet d'avoir la déconnexion disponible et si l'utilisateur connecté avec un rôle d'administrateur, un bouton « Menu administrateur ». La récupération des produits me permet d'afficher la liste de produit dans les différentes pages où ils sont appelés (index, boutique, panier).

```
menu.jsp

<!-- Importing necessary classes -->
<%@page import="com.nanasenseimvc.connection.DbCon"%>
<%@page import="com.nanasenseimvc.dao.ProductDao"%>
<%@page import="com.nanasenseimvc.model.Product"%>
<%@page import="java.util.List"%>
<%@page import="com.nanasenseimvc.model.User"%>

<!-- Header start -->
<header role="banner">
    <div class="container1">
        <!-- Logo with link to home page -->
        <div>
            <a href="index.jsp"> </a>
        </div>
        <!-- Main navigation -->
        <nav class="navigation">
            <!-- Burger menu for mobile version -->
            <div class="burger-menu">
                <span class="burger-icon">
                    <span></span>
                    <span></span>
                    <span></span>
                </span>
                <span class="close-icon">&#x2715;</span>
            </div>
            <!-- Navigation links list -->
            <ul>
                <li class="cours"><a href="cours.jsp">Cours</a></li>
                <li class="boutique"><a href="boutique.jsp">Boutique</a></li>
                <li class="blog"><a href="blog.jsp">Blog</a></li>
                <li>
                    <!-- Retrieving products and authenticated user -->
                    ProductDao prodd = new ProductDao(DbCon.getConnection());
                    List<Product> products = prodd.getAllProducts();
                    User auth = (User)request.getSession().getAttribute("auth");
                    if(auth !=null){
                        <!-- Display connected user's username -->
                        <li class="connexion"><a href="account.jsp"><%= auth.getPseudo() %></a>
                        </li>
                        <!-- Logout button -->
                        <li><a href="logout"></a></li>
                        <!-- Display admin menu link if user is an admin -->
                        < if(auth.getRoles().equals("admin")) { %>
                            <li><a href="%= request.getContextPath()
%>/admin/adminHome.jsp">Admin Menu</a></li>
                        < %>
                    } else { %>
                        <!-- Display login link if user is not connected -->
                        <li class="connexion"><a href="login.jsp">Connexion</a></li>
                    } %>
                </li>
            </ul>
        </nav>
    </div>
</header>
```

Il possède un CSS commun au footer appelé dans le head.jsp ce dernier gère aussi des média queries. C'est dans ce fichier CSS que j'ai intégré le root qui me permet de créer des variables utilisables dans l'ensemble du projet. Cela facilitera le travail si un changement de couleur est réalisé un jour.

```
header-footer.css

:root{
  --blackcours : rgb(55,62,105);
  --flashcours : rgb(101,113,187);
  --pastelcours : rgb(156,165,244);
  --lightcours : rgb(237,238,255);

  --blackboutique : rgb(117,69,56);
  --flashboutique : rgb(246, 166, 151);
  --pastelboutique : rgb(254, 197, 186);
  --lightboutique : rgb(252, 243, 241);

  --blackblog : rgb(124, 105, 40);
  --flashblog : rgb(249, 232, 99);
  --pastelblog : rgb(248, 232, 180);
  --lightblog : rgb(255, 248, 226);

  --blackconnexion : rgb(21, 97, 71);
  --flashconnexion : rgb(45, 179, 133);
  --pastelconnexion : rgb(125, 222, 199);
  --lightconnexion : rgb(218, 252, 240);

  @font-face {
    font-family: 'Sailors';
    src: url('fonts/Sailors-1.eot');
    src: url('fonts/Sailors-1.eot?#iefix') format('embedded-opentype'),
        url('fonts/Sailors-1.woff2') format('woff2'),
        url('fonts/Sailors-1.woff') format('woff'),
        url('fonts/Sailors-1.ttf') format('truetype'),
        url('fonts/Sailors-1.svg#Sailors') format('svg');
  }
}
```

Root avec les variables CSS

Style du menu de la version PC

```
header-footer.css

/* Header */
.container1{
  display: flex;
  flex-wrap: nowrap;
  align-items:center;
  max-width: 100%;
  justify-content: space-around;
  background : linear-gradient(90deg ,var(--pastelcours)0%, var(--pastelboutique) 33%, var(--pastelblog)66%, var(--pastelconnexion)86%);
  padding : 10px;
}
.logo{
  display: block;
  position: relative;
  top : 1rem;
  width:150px;
  height:150px;
}
.navigation{
  display: flex;
  flex-grow : 1;
  max-width:60%;
  justify-content : center;
}
.navigation ul {
  display: flex;
  list-style-type: none;
  margin: 0;
  padding: 0;
}
.navigation ul li {
  margin-left: 10px;
  a{
    padding : 1rem;
    text-decoration: none;
    font-family: 'cherry Bomb One';
    letter-spacing: 1px;
    text-align: center;
    transition: all 0.3s ease;
  }
}
```

```

header-footer.css

.burger-menu {
  display: none;
  cursor: pointer;
  position: relative;
  width: 30px;
  height: 30px;
}

.burger-icon, .close-icon {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  transition: opacity 0.3s ease;
}

.burger-icon span {
  display: block;
  width: 100%;
  height: 3px;
  background-color: #333;
  margin: 0px 0;
  transition: 0.4s;
}

.close-icon {
  opacity: 0;
  font-size: 30px;
  text-align: center;
  line-height: 30px;
}

.burger-menu.active .burger-icon {
  opacity: 0;
}

.burger-menu.active .close-icon {
  opacity: 1;
}

```

Style du menu burger de la version PC

```

header-footer.css

@media screen and (max-width: 1080px) and (min-width: 750px){
  .cours {
    margin-right: 1rem ;
  }
  .boutique{
    margin-right: 1rem ;
  }
  .blog {
    margin-right: 1rem;
  }
}

@media screen and (max-width: 750px) {
  .container1{
    display: flex;
    flex-wrap: nowrap;
    align-items: center;
    justify-content: space-around;
    background: linear-gradient(90deg, var(--pastelcours)0%, var(--pastelboutique) 33%, var(--pastelblog)66%, var(--pastelconnexion)86%);
    padding: 10px;
  }
  .logo{
    display: block;
    position: relative;
    top: 0.5rem;
    width: 75px;
    height: 75px;
  }
  .navigation{
    display: flex;
    flex-grow: 1;
    max-width: 25%;
    justify-content: flex-end;
  }
  .navigation ul {
    display: none;
    flex-direction: column;
    position: absolute;
    top: 100px;
    right: 0;
    width: 50%;
  }
  .navigation ul.active {
    display: flex;
  }
  .navigation ul li {
    margin: 0;
    text-align: center;
    padding: 10px 0;
  }
  .burger-menu {
    display: flex;
  }
}

main{
  background: none;
}

```

```

header-footer.css

.cours {
  background-color: var(--lightcours);
  border-radius: 50px;
  border: 2px solid, var(--flashcours);
  margin-right: 2rem ;
  padding: 3px 5px 6px 5px;
  a {
    color: var(--blackcours);
  }
}

.navigation ul li.cours:hover, .navigation ul li.cours.active{
  border-radius: 50px;
  background-color: var(--blackcours);
}

.navigation ul li:hover a, .navigation ul li.active a {
  color: white;
}

.boutique{
  background-color: var(--lightboutique);
  border-radius: 50px;
  border: 2px solid, var(--flashboutique);
  margin-right: 2rem ;
  padding: 3px 5px 6px 5px;
  a {
    color: var(--blackboutique);
  }
}

.navigation ul li.boutique:hover, .navigation ul li.boutique.active{
  border-radius: 50px;
  background-color: var(--blackboutique);
}

.blog {
  background-color: var(--lightblog);
  border-radius: 50px;
  border: 2px solid, var(--flashblog);
  padding: 3px 5px 6px 5px;
  margin-right: 20%;
  a {
    color: var(--blackblog);
  }
}

.navigation ul li.blog:hover, .navigation ul li.blog.active {
  border-radius: 50px;
  background-color: var(--blackblog);
}

.logged{
  display: flex;
  flex-direction: row;
  flex-wrap: nowrap;
  align-items: center;
}

.connexion {
  background-color: var(--lightconnexion);
  border-radius: 50px;
  border: 2px solid, var(--flashconnexion);
  padding: 3px 5px 6px 5px;
  a {
    color: var(--blackconnexion);
    padding: 1rem;
    text-decoration: none;
    font-family: 'cherry Bomb One';
    letter-spacing: 1px;
    text-align: center;
  }
}

.navigation ul li.connexion:hover, .navigation ul li.connexion.active{
  border-radius: 50px;
  background-color: var(--blackconnexion);
}

.logout{
  margin-left: 1rem;
  width: 26px;
  height: 26px;
}

```

Définition du style des boutons du menu ainsi que leur hover.

Media queries qui permettent au menu d'être responsive.

Pour intégrer du dynamisme dans mon menu un script JavaScript est intégré dans le footer grâce à une balise `<script> <%@ include file="../JavaScript/menu.js"%>` `</script>` dans le footer.

```
menu.js

// Wait for the DOM to be fully loaded before executing the script
document.addEventListener('DOMContentLoaded', function() {
  // Select the burger menu element
  const burgerMenu = document.querySelector('.burger-menu');
  // Select the navigation list
  const nav = document.querySelector('.navigation ul');

  // Add click event listener to the burger menu
  burgerMenu.addEventListener('click', function() {
    // Toggle 'active' class on the burger menu
    this.classList.toggle('active');
    // Toggle 'active' class on the navigation list
    nav.classList.toggle('active');
  });
});

// Wait for the DOM to be fully loaded before executing the script
document.addEventListener('DOMContentLoaded', function() {
  // Select all menu links
  const menuLinks = document.querySelectorAll('.navigation ul li a');

  // Get the current page path
  const currentPath = window.location.pathname;

  menuLinks.forEach(link => {
    // Get the href attribute of the link
    const href = link.getAttribute('href');
    // Check if the link's href matches the current path
    if (currentPath.includes(href) && href !== '') {
      // Add 'active' class to the parent element (li) of the matching
      link
      link.parentElement.classList.add('active');
    } else {
      // Remove 'active' class from other elements
      link.parentElement.classList.remove('active');
    }
  });
  // Add click event listener to each link
  link.addEventListener('click', function() {
    // Remove 'active' class from all menu items
    menuLinks.forEach(item =>
    item.parentElement.classList.remove('active'));
    // Add 'active' class to the clicked item's parent
    this.parentElement.classList.add('active');
  });
});
});
```

Ce script Javascript permettant pour la première partie d'activer le menu hamburger et de pouvoir interagir avec lui. Et dans la deuxième partie pouvoir ajouter une classe 'active' au lien qui se situe dans le menu en fonction de la page consultée.

Le footer contient un rappel du menu, les liens vers les réseaux sociaux, l'inscription à la newsletter ainsi que tous les liens utiles par rapport au site et son activité : les mentions légales, la politique de gestion des données, les conditions générales de ventes. L'apparition du formulaire pour la newsletter est conditionnée par le rôle de l'utilisateur. Il est visible pour les utilisateurs non connectés, pour les clients et les utilisateurs connectés. Celui-ci est remplacé par un lien vers le menu administrateur quand un administrateur est connecté.

```

<!-- Footer section of the website -->
<footer role="contentinfo" id="footer">
  <div class="footer">
    <!-- Logo section with link to homepage -->
    <div class="logo"> <a href="index.jsp">  </a> </div>
    <!-- Footer navigation menu -->
    <nav aria-label="Menu de pied de page" id="navfooter">
      <ul>
        <li class="menu"><a href="cours.jsp">Cours</a></li>
        <li class="menu"><a href="boutique.jsp">Boutique</a></li>
        <li class="menu"><a href="blog.jsp">Blog</a></li>
        <li class="menu"><a href="/Contact/contact.html">Contact</a></li>
      </ul>
    </nav>

    <!-- Social media links -->
    <div class="rs">
      <a href="https://www.instagram.com/koisandrawing/" aria-label="Instagram de Nana-
sensei">
        
      </a>
      <a href="https://www.tiktok.com/@koisandrawing?_t=8nzbH4Lb6Aq&_r=1" aria-label="TikTok
de Nana-sensei">
        
      </a>
      <a href="https://www.redbubble.com/fr/people/Koi-san/shop?utm_source=fb-native-
app&utm_campaign=share-artist&utm_medium=android" aria-label="Boutique Redbubble de Nana-
sensei">
        
      </a>
    </div>

    <!-- Conditional rendering based on user role -->
    <% if(auth.getRoles().equals("admin")) { %>
      <!-- Admin menu link for admin users -->
      <a href="<%= request.getContextPath() %>/admin/adminHome.jsp">Admin Menu</a>
    <% } else { %>
      <!-- Newsletter subscription form for non-admin users -->
      <div class="newsletter">
        <h3>Newsletter</h3>
        <form>
          <div class="input">
            <label class="label" for="email">Email address</label>
            <input id="email" class="adressesmail" type="email" name="mail"
required placeholder="Email address">
          </div>
          <button class="ctaconnexion" type="submit">Subscribe</button>
        </form>
      </div>
    <% } %>
  </div>

  <!-- Legal information and links -->
  <p class="legal">
    5copy; 2024 Nana-sensei | Tous droits réservés |
    <a href="<%= request.getContextPath() %>/mentionsLegales.jsp">Mentions légales</a> |
    <a href="#">Conditions générales de vente</a> |
    <a href="#">Politique de confidentialité</a>
  </p>
</footer>
<script>
<%@ include file = "../JavaScript/menu.js"%>
</script>

```

Ce footer a également une partie CSS responsive grâce à des media queries et contient le script du menu qui est applicable sur toutes les pages du site internet.

```

/* Footer */
#footer{
  background: linear-gradient(90deg, var(--pastelcours)0%, var(--pastelboutique) 33%, var(--pastelblog)66%, var(--pastelconnexion)86%);
}
.footer {
  display: flex;
  flex-wrap: nowrap;
  align-items: center;
  max-width: 100%;
  justify-content: space-around;
  margin-top: 2rem;
  margin-bottom: 2rem;
}
ul{
  padding: 0;
}
.menu {
  list-style: none;
  line-height: 1.5;
}
.menu a {
  text-decoration: none;
  color: var(--blackboutique);
  font-family: 'cherry bomb one';
  letter-spacing: 1px;
  font-weight: bold;
}
.rs{
  display: flex;
  flex-direction: column;
}
.newsletter {
  h2{
    text-align: center;
    color: var(--blackconnexion);
    font-family: 'cherry bomb one';
    letter-spacing: 1px;
    margin-bottom: 1rem;
    margin-top: 1rem;
  }
  form{
    display: flex;
    flex-direction: column;
    align-items: flex-end;
  }
  .input{
    display: flex;
    flex-direction: column;
  }
  .label{
    color: var(--blackconnexion);
    font-family: 'Sailors';
    font-size: 12px;
    padding-left: 1rem;
  }
  .adressemail{
    border-radius: 50px;
    border: 2px, solid, var(--flashconnexion);
    background-color: var(--lightconnexion);
    margin-bottom: 1rem;
    padding: 5px;
  }
}
::placeholder {
  color: var(--blackconnexion);
  font-family: 'Sailors';
  font-size: 12px;
  padding-left: 1rem;
}
.ctaconnexion{
  border-radius: 50px;
  border: 2px, solid, var(--flashconnexion);
  background-color: var(--lightconnexion);
  margin-left: 117px;
  text-align: center;
  color: var(--blackconnexion);
  font-family: 'cherry bomb one';
  letter-spacing: 1px;
  padding: 3px 5px 6px 7px;
  width: fit-content;
}
}
#navfooter{
  width: 100%;
}
.legal{
  text-align: center;
  padding-bottom: 1rem;
  font-family: 'Sailors';
  font-size: 12px;
}
a{
  text-decoration: none;
  color: var(--blackboutique);
}
}

```

```

@media screen and (max-width: 550px){
  .footer {
    display: flex;
    flex-direction: column-reverse;
    align-items: center;
    max-width: 100%;
    justify-content: space-around;
    margin-top: 2rem;
    margin-bottom: 2rem;
  }
  #navfooter{
    width: 70%;
  }
  #navfooter ul{
    display: flex;
    justify-content: space-evenly;
  }
  .newsletter form{
    display: flex;
    flex-direction: row;
    align-items: center;
  }
  .newsletter form .ctaconnexion{
    border-radius: 50px;
    border: 2px, solid, var(--flashconnexion);
    background-color: var(--lightconnexion);
    margin-left: 1rem;
    text-align: center;
    color: var(--blackconnexion);
    font-family: 'cherry bomb one';
    letter-spacing: 1px;
    padding: 3px 5px 6px 7px;
    width: fit-content;
  }
  .rs{
    width: 80%;
    display: flex;
    flex-direction: row;
    justify-content: space-evenly;
    flex-grow: 1;
  }
  .legal{
    padding: 1rem;
    justify-items: center;
    text-align: center;
    padding-bottom: 1rem;
    font-family: 'Sailors';
    font-size: 12px;
  }
  a{
    text-decoration: none;
    color: var(--blackboutique);
  }
}

```

Media queries pour la responsivité du site

CSS du footer en version PC

Non compris dans le dossier « includes » mais dans le dossier « Java » et pourtant utilisé dans toutes les pages de ce site, se trouve une classe java qui est le lien avec la base de donnée. Elle est utilisée également dans le menu puisque sans celle-ci, aucune connexion à la base de données n'est possible donc pas de récupération d'information.

A screenshot of a code editor window titled 'DbCon.java'. The code is written in Java and defines a package 'com.nanasenseimvc.connection', imports 'java.sql.Connection', 'java.sql.DriverManager', and 'java.sql.SQLException'. It then defines a public class 'DbCon' with a static 'Connection' object 'connection' initialized to null. A static method 'getConnection()' is defined, which checks if the connection is null, loads the MySQL JDBC driver, establishes a connection to a database named 'nana_sensei' on localhost, and prints a success message to the console before returning the connection. The password field in the connection string is redacted with a black box.

```
package com.nanasenseimvc.connection;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DbCon {

    // Static Connection object to be reused
    private static Connection connection = null;

    // Method to get database connection
    public static Connection getConnection() throws ClassNotFoundException, SQLException {
        // Check if connection is null (not established yet)
        if(connection == null) {
            // Load the MySQL JDBC driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Establish connection to the database
            // URL format: jdbc:mysql://host:port/database_name
            connection =
            DriverManager.getConnection("jdbc:mysql://localhost:3306/nana_sensei", "root", " ");

            // Print success message to console
            System.out.print("je suis connecté à la database! ");
        }
        // Return the established connection
        return connection;
    }
}
```

Lorsque je lance mon projet, la mention « Je suis connecté à la database ! » apparaît dans la console grâce à la commande « System.out.print(« je suis connecté à la database ! »); ».

b) La page d'accueil

Sur cette page d'accueil sont présentés brièvement les différentes parties du site avec un aperçu de chaque. Lorsque le site sera opérationnel, une section pour le blog et des témoignages d'élèves seront ajoutés. Du code Java est utilisé dans la section « boutique » afin de limiter le nombre d'article affiché sur la page d'accueil.

```

index.jsp

<%@page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>

<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Nano-sensei | Accueil</title>

<!-- Include head.jsp file -->
<%@include file="includes/head.jsp" %>

<!-- Include CSS files -->
<style>
    <%@include file="CSS/styleindex.css"%>
    <%@include file="CSS/styleIndexMobile.css"%>
</style>

</head>
<body>

<!-- Include menu.jsp file -->
<%@include file="includes/menu.jsp" %>

    <main role="main">
        <!-- Hero section -->
        <section role="region" class="hero" aria-labelledby="hero-title">

            <div class="hero">
                <div class="text">
                    <p>Envie d'apprendre à dessiner ? <br>
                        Partager ses connaissances avec des passionnées ? <br>
                        Obtenir des design originaux et des ebooks de qualité ?
                    </p>
                    <div class="titrehero">
                        <div>
                            <h1 id="hero-title">REJOINS NANA- <br> SENSEI DANS
                                SON<br> UNIVERS</h1>
                        </div>
                        <div class="photopro"></div>
                    </div>
                    <div>
                        <!-- Button to redirect to boutique.jsp -->
                        <button class="btnndraw"
                            onclick="window.location.href='boutique.jsp'"> Let's Draw</button>
                    </div>
                    <div>
                        <div class="letsdraw">
                    </div>
                </div>
            </div>
        </section>
    </main>

```

```

<!-- Cours section -->
<section role="region" class="lescours" aria-labelledby="titrelecours">
  <div class="containercours">
    <h2 id="titrelecours"><a href="cours.jsp">Les cours</a></h2>
    <div class="firstline">
      <div class="textcours">Des cours collectifs de dessins en
présentiel on tant que prestataire
dans les centres sociaux et autres lieux proposant de activités
à <br>
      Evry <br>
      Chatenay Malabry <br>
      Choisyleroi</div>
      <div class="illust"> </div>
    </div>
  </section>
<!-- Shop section -->
<section role="region" class="laboutique" aria-
labelledby="titreboutique">
  <div class="containerboutique">
    <h2 id="titreboutique"><a href="boutique.jsp">La boutique</a>
</h2>
    <article role="article">
      <%ProductDao prod = new ProductDao(DbCon.getConnection());
      List<Product> product = prod.getAllProducts();
      // Check if the products list is not empty
      if(!products.isEmpty()){
        // Initialize a counter to keep track of displayed products
        int count = 0;
        // Iterate through each product in the products list
        for (Product p : products){
          // Check if we have displayed less than 3 products
          if(count < 3) {
            <!-- Start of product card HTML structure -->
            <div class="productcard">
              <!-- Link to the product detail page -->
              <a href="productboutique.jsp?id=<%p.getId()%>">
                <!-- Container for the product image -->
                <div class="photocard">
                  <!-- Display the product image -->
                  
                </div>
                <!-- Display the product name -->
                <h2 class="nomproduit"><%p.getName()%></h2>
                <!-- Display the product category -->
                <p class="descriptionproduit"><%p.getCategory()%></p>
              </div>
            <!-- End of product card HTML structure -->
            <%
              // Increment the counter after displaying a product
              count++;
            } else {
              // If we've already displayed 3 products, exit the loop
              break;
            }
          }
        }
      }
    </article>
  </div>
</section>
</main>
<!-- Include footer.jsp file -->
<%include file="includes/footer.jsp" %>
</body>
</html>

```

Structure HTML/Java de la page d'accueil !

Pour personnaliser cette page d'accueil, du style est intégré via des fichiers CSS externes.

Pour gérer les performances, le code est minifié. La minification CSS réduit la taille des fichiers CSS afin qu'ils se chargent plus rapidement. La minification CSS fonctionne en éliminant tous les caractères et espaces inutiles du balisage CSS sans affecter la façon dont les navigateurs l'interprètent.


```

StyleIndex.css

/*HERO style*/
.hero {
  width: 100%;
  background: linear-gradient(90deg, var(--pastelcours)0%, var(--pastelboutique) 33%, var(--pastelblog)66%, var(--pastelconnexion)86%);
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  align-items: center;
}
.text {
  width: 100%;
  padding-left: 6rem;
  padding-top: 3rem;
  p {
    background: -webkit-linear-gradient(left, var(--blackcours)0%, var(--blackboutique) 50%, var(--blackblog)80%);
    background-clip: text;
    -webkit-background-clip: text;
    color: transparent;
    font-family: 'cherry Bomb One';
    letter-spacing: 1px;
    text-align: center;
  }
}
.titrehero {
  width: 100%;
  padding-top: 3rem;
  display: flex;
  flex-direction: row;
  flex-wrap: nowrap;
  justify-content: space-evenly;
  align-items: center;
}
h1{
  background: -webkit-linear-gradient(left, var(--blackcours)0%, var(--blackboutique) 80%);
  background-clip: text;
  -webkit-background-clip: text;
  color: transparent;
  font-family: 'cherry Bomb One';
  letter-spacing: 1px;
  text-align: center;
}
.letsdraw{
  width: 95%;
  height: 100%;
}
.btdraw{
  background-color: transparent;
  border-radius: 50px;
  border: 2px solid var(--blackcours);
  margin-top: 1rem;
  margin-bottom: 1rem;
  margin-left: 45%;
  background: -webkit-linear-gradient(left, var(--blackcours)0%, var(--blackboutique) 80%);
  background-clip: text;
  -webkit-background-clip: text;
  color: transparent;
  font-family: 'cherry Bomb One';
  letter-spacing: 1px;
  transition: all 0.3s ease; /* Ajoute une transition fluide */
}
.btdraw:hover {
  background-clip: border-box;
  -webkit-background-clip: border-box;
  color: white;
}

```

```

StyleIndex.css

/*Les cours style*/
.containercours{
  margin: 2rem;
  background-color: var(--lightcours);
  border-radius: 50px;
  padding: 2rem;
  display: flex;
  flex-direction: column;
}
#titre{
  text-decoration: none;
  margin: 1rem 2rem 1rem 2rem;
  width: fit-content;
  border: solid;
  border-radius: 50px;
  border-color: var(--blackcours);
  background-color: var(--pastelcours);
  padding: 0.5rem 1rem 0.5rem 1rem;
  a {
    text-decoration: none;
    font-family: 'cherry Bomb One';
    letter-spacing: 1px;
    text-align: center;
    color: var(--blackcours);
  }
}
.firstline{
  width: 80%;
  display: flex;
  justify-content: space-around;
  margin: auto;
  align-items: center;
}
.textcours{
  width: 50ch;
  padding: 1rem;
  font-family: 'Sailors';
  text-align: center;
  color: var(--blackcours);
}
.illustr{
  max-width: 80%;
  max-height: 80%;
}

```

CSS de la page index.jsp

```

StyleIndex.css

/*La boutique style*/
article{
  width: 80%;
  margin: auto;
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
}
.containerboutique{
  margin: 2rem;
  background-color: var(--lightboutique);
  border-radius: 50px;
  padding: 2rem;
  display: flex;
  flex-direction: column;
}
#titreboutique {
  text-decoration: none;
  margin: 2rem;
  width: fit-content;
  border: solid;
  border-radius: 50px;
  border-color: var(--blackboutique);
  background-color: var(--pastelboutique);
  padding: 0.5rem 1rem 0.5rem 1rem;
  a {
    text-decoration: none;
    font-family: 'cherry Bomb One';
    letter-spacing: 1px;
    text-align: center;
    color: var(--blackboutique);
  }
}

```

```

StyleIndex.css

.productcard {
  margin: 1rem;
  max-width: 25%;
  border: 2px solid var(--flashboutique);
  border-radius: 50px;
  background-color: var(--lightboutique);
  padding: 1rem;
  color: var(--blackboutique);
  flex: 0 0 30%;
  min-width: 25%;
  display: flex;
  flex-direction: column;
  justify-content: center;
  a {
    text-decoration: none;
    text-align: center;
    color: var(--blackboutique);
    display: flex;
    flex-direction: column;
  }
}
.photocard{
  width: 50%;
  align-self: center;
  margin-bottom: 1rem;
}
.nomproduit{
  max-width: 30ch;
  font-family: 'cherry bomb one';
  letter-spacing: 1px;
  margin-bottom: 1rem;
}
.descriptionproduit{
  max-width: 30ch;
  font-family: 'Sailors';
  letter-spacing: 1px;
  margin-bottom: 1rem;
}
.prix{
  text-align: right;
  font-family: 'Sailors';
  letter-spacing: 1px;
}

```

```

StyleIndexResponsive.css

@media screen and (max-width: 800px){
/*HERO style*/
.hero {
width: 100%;
background: linear-gradient(90deg, var(--pastelcours)0%, var(--pastelboutique) 33%, var(--pastelblog)66%, var(--pastelconnexion)86%);
height: 50vh;
}
.text {
width: 86%;
background: -webkit-linear-gradient(left, var(--blackcours)0%, var(--blackboutique) 50%, var(--blackblog)89%);
background-clip: text;
-webkit-background-clip: text;
color: transparent;
font-family: 'cherry Bomb One';
letter-spacing: 1px;
text-align: center;
padding: 0;
margin: auto;
}
.titrehero {
width: 100%;
padding: 3rem;
display: flex;
flex-direction: row;
flex-wrap: nowrap;
justify-content: space-evenly;
align-items: center;
}
h1{
background: -webkit-linear-gradient(left, var(--blackcours)0%, var(--blackboutique) 89%);
background-clip: text;
-webkit-background-clip: text;
color: transparent;
font-family: 'cherry Bomb One';
letter-spacing: 1px;
text-align: center;
}
.letsdraw{
display:none;
}
.btndraw{
background-color: transparent;
border-radius: 50px;
border: 2px solid var(--blackcours);
margin:auto;
background: -webkit-linear-gradient(left, var(--blackcours)0%, var(--blackboutique) 89%);
background-clip: text;
-webkit-background-clip: text;
color: transparent;
font-family: 'cherry Bomb One';
letter-spacing: 1px;
}
}

```

```

StyleIndexResponsive.css

/*Les cours style*/
.containercours{
margin: 2rem;
background-color: var(--lightcours);
border-radius: 50px;
padding: 2rem;
display: flex;
flex-direction: column;
}
#titreccours{
text-decoration: none;
margin: 1rem 2rem 1rem 2rem;
width: fit-content;
border: solid;
border-radius: 50px;
border-color: var(--blackcours);
background-color: var(--pastelcours);
padding: 0.5rem 1rem 0.5rem 1rem;
a {
text-decoration: none;
font-family: 'cherry Bomb One';
letter-spacing: 1px;
text-align: center;
color: var(--blackcours);
}
}
.firstline{
width: 80%;
display: flex;
justify-content: space-around;
margin: auto;
align-items: center;
}
.textccours{
width: 50ch;
padding: 1rem;
font-family: 'Sailors';
text-align: center;
color: var(--blackcours);
}
.illustr{
max-width: 80%;
max-height: 80%;
display: none;
}
}

```

CSS responsive de la page index.jsp

```

StyleIndexResponsive.css

/*La boutique style*/
article{
width: 100%;
margin: auto;
display: flex;
flex-direction: column;
flex-wrap: wrap;
justify-content: center;
}
.containerboutique{
margin: 2rem;
background-color: var(--lightboutique);
border-radius: 50px;
padding: 2rem;
display: flex;
flex-direction: column;
}
#titreboutique {
text-decoration: none;
margin: 2rem;
width: fit-content;
border: solid;
border-radius: 50px;
border-color: var(--blackboutique);
background-color: var(--pastelboutique);
padding: 0.5rem 1rem 0.5rem 1rem;
a {
text-decoration: none;
font-family: 'cherry Bomb One';
letter-spacing: 1px;
text-align: center;
color: var(--blackboutique);
}
}
}

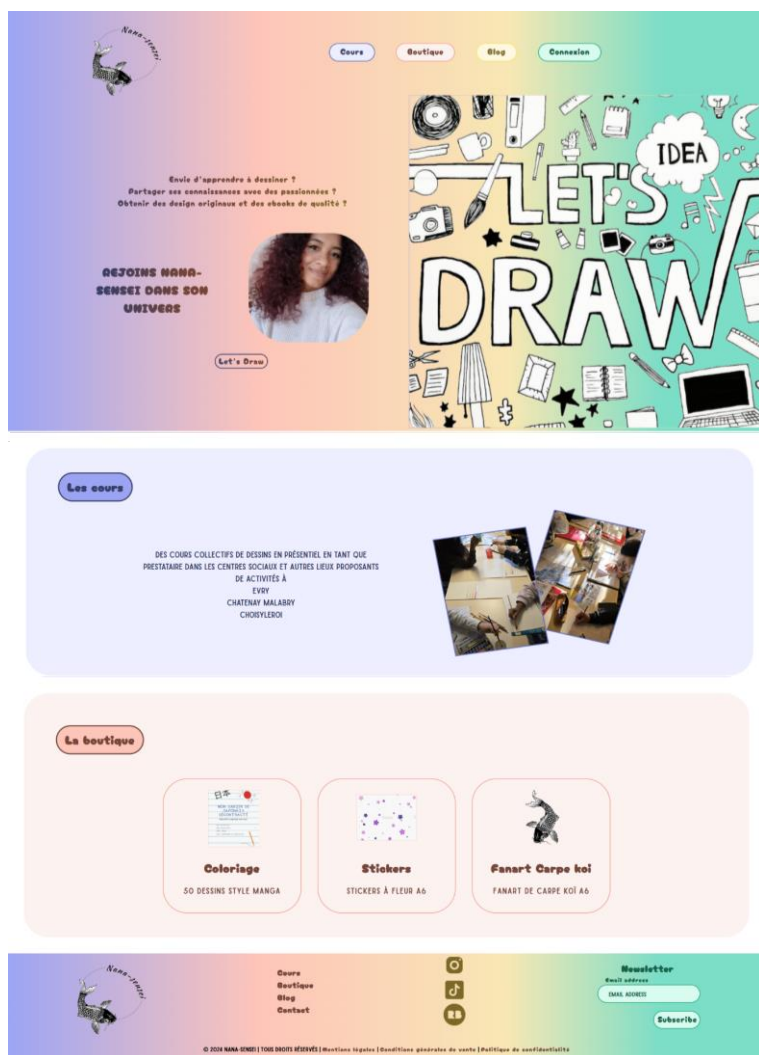
```

```

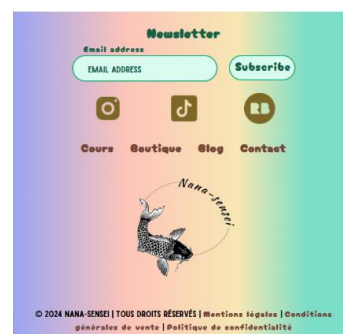
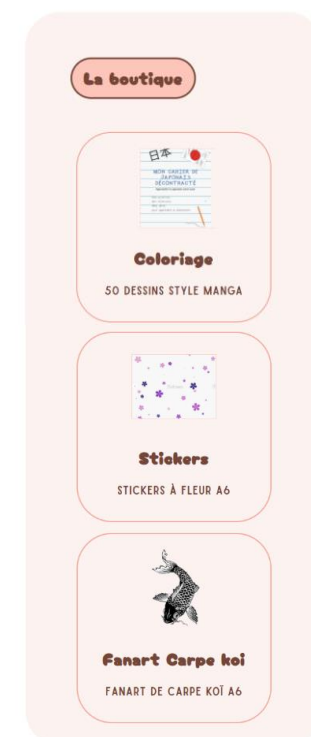
StyleIndexResponsive.css

.productcard {
margin: auto;
margin-top: 15px;
max-width: 80%;
border: 2px solid var(--flashboutique);
border-radius: 50px;
background-color: var(--lightboutique);
padding: 1rem;
color: var(--blackboutique);
flex: 0 0 30%;
min-width: 30%;
display: flex;
flex-direction: column;
justify-content: center;
a{
text-decoration: none;
text-align: center;
color: var(--blackboutique);
display: flex;
flex-direction: column;
}
}
.photocard{
width: 50%;
align-self: center;
margin-bottom: 1rem;
}
.nomproduit{
max-width: 30ch;
font-family: 'cherry bomb one';
letter-spacing: 1px;
margin-bottom: 1rem;
}
.descriptionproduit{
max-width: 30ch;
font-family: 'Sailors';
letter-spacing: 1px;
margin-bottom: 1rem;
}
.prix{
text-align: right;
font-family: 'Sailors';
letter-spacing: 1px;}
}

```



Vue de la page d'accueil version PC



Vue de la page d'accueil version mobile

c) La sécurité

```
Encryption.java

package com.nanasenseimvc.model;

import java.io.UnsupportedEncodingException;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;

public class Encryption {
    // Converts Bytes to String
    private static String convertToHex(byte[] data) {
        StringBuffer buf = new StringBuffer(); // StringBuffer class
        // Similar to String class, but can be modified
        for (int i = 0; i < data.length; i++) {
            // The >>> operator is an unsigned right shift operator in Java
            // It divides the operand by 4 to the power of the right operand (0x0F)
            int halfByte = (data[i] >>> 4) & 0x0F;
            int two_halfs = 0;
            do {
                // If halfByte is between 0 and 9
                if ((0 <= halfByte) && (halfByte <= 9)) {
                    // Append the character sequence from 0 to 9
                    buf.append((char) ('0' + halfByte));
                } else {
                    // Append the character sequence from a to e
                    buf.append((char) ('a' + (halfByte - 10)));
                }
                halfByte = data[i] & 0x0F;
            } while (two_halfs++ < 1);
        }
        return buf.toString();
    }
    // Hashing
    public static String getSHA1(String text) throws NoSuchAlgorithmException,
        UnsupportedEncodingException {
        // This class provides applications with the functionality of a message digest
        algorithm (e.g., SHA-1 or SHA-256)
        MessageDigest md;
        // Choose an algorithm
        md = MessageDigest.getInstance("SHA-1");
        // Create a byte array of 40 characters
        byte[] sha1hash = new byte[40];
        md.update(text.getBytes("iso-8859-1"), 0, text.length());
        // Complete the hash calculation by performing final operations such as padding
        sha1hash = md.digest();
        return convertToHex(sha1hash);
    }
}
```

Cette classe « Encryption.java » a pour but principal de fournir une méthode pour hacher une chaîne de caractères en utilisant l'algorithme SHA-1.

La première méthode privée permet un tableau d'octet en chaîne de caractère hexadécimal. La deuxième méthode publique cette fois prend une chaîne de caractères en entrée et renvoie son hash SHA-1.

Cette classe sera utilisée pour la sécurisation du mot de passe des utilisateurs, ainsi même en ayant accès à la base de données le mot de passe sera protégé.

Une autre sécurité apportée au site est l'utilisation des requêtes préparées afin d'empêcher les injections SQL.

d) La connexion

Les modèles

Ces modèles définissent les informations nécessaires à mes utilisateurs qui peuvent utiliser certaines fonctionnalités sans informations supplémentaires. Une classe java « Client » a été créée et héritée pour les clients de la boutique et pour les futures réservations de cours.

```
package com.nanasenseimvc.model;

// User class to represent a user in the system
public class User {
    // User attributes
    private int id;
    private String email;
    private String password;
    private String pseudo;
    private String securityQuestion;
    private String answer;
    private String roles;

    // Default constructor
    public User() {
        super();
    }

    // Parameterized constructor
    public User(int id, String email, String password, String pseudo,
        String securityQuestion, String answer, String roles) {
        super();
        this.id = id;
        this.email = email;
        this.password = password;
        this.pseudo = pseudo;
        this.securityQuestion = securityQuestion;
        this.answer = answer;
        this.roles = roles;
    }

    // Getter for id
    public int getId() {
        return id;
    }

    // Setter for id
    public void setId(int id) {
        this.id = id;
    }

    // Getter for email
    public String getEmail() {
        return email;
    }

    // Setter for email
    public void setEmail(String email) {
        this.email = email;
    }

    // Getter for password
    public String getPassword() {
        return password;
    }

    // Setter for password
    public void setPassword(String password) {
        this.password = password;
    }

    // Getter for pseudo (username)
    public String getPseudo() {
        return pseudo;
    }

    // Setter for pseudo (username)
    public void setPseudo(String pseudo) {
        this.pseudo = pseudo;
    }

    // Getter for security question
    public String getSecurityQuestion() {
        return securityQuestion;
    }

    // Setter for security question
    public void setSecurityQuestion(String securityQuestion) {
        this.securityQuestion = securityQuestion;
    }

    // Getter for security question answer
    public String getAnswer() {
        return answer;
    }

    // Setter for security question answer
    public void setAnswer(String answer) {
        this.answer = answer;
    }

    // Setter for user roles
    public void setRoles(String roles) {
        this.roles = roles;
    }

    // Getter for user roles
    public String getRoles() {
        return roles;
    }
}
```

```
package com.nanasenseimvc.model;

// Client class that extends the User class, representing a client in
the system
public class Client extends User {
    // Additional attributes specific to a client
    private String nom; // Last name
    private String prenom; // First name
    private String email; // Email
    private String adressePostale; // Postal address
    private String cp; // Postal code
    private String ville; // City
    private String pays; // Country
    private String telephone; // Phone number

    // Default constructor
    public Client() {
        super();
    }

    // Constructor with client-specific attributes
    public Client(String nom, String prenom, String email, String
        adressePostale, String cp, String ville, String pays,
        String telephone) {
        super();
        this.nom = nom;
        this.prenom = prenom;
        this.email = email;
        this.adressePostale = adressePostale;
        this.cp = cp;
        this.ville = ville;
        this.pays = pays;
        this.telephone = telephone;
    }

    // Getters and setters for client-specific attributes
    public String getNom() {
        return nom;
    }

    public void setNom(String nom) {
        this.nom = nom;
    }

    public String getPrenom() {
        return prenom;
    }

    public void setPrenom(String prenom) {
        this.prenom = prenom;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getAdressePostale() {
        return adressePostale;
    }

    public void setAdressePostale(String adressePostale) {
        this.adressePostale = adressePostale;
    }

    public String getCp() {
        return cp;
    }

    public void setCp(String cp) {
        this.cp = cp;
    }

    public String getVille() {
        return ville;
    }

    public void setVille(String ville) {
        this.ville = ville;
    }

    public String getPays() {
        return pays;
    }

    public void setPays(String pays) {
        this.pays = pays;
    }

    public String getTelephone() {
        return telephone;
    }

    public void setTelephone(String telephone) {
        this.telephone = telephone;
    }

    // Constructor that calls the superclass (User) constructor
    public Client(int id, String email, String password, String pseudo,
        String securityQuestion, String answer,
        String roles) {
        super(id, email, password, pseudo, securityQuestion, answer,
            roles);
    }

    // Override toString() method to provide a string representation of
the Client object
    @Override
    public String toString() {
        return "Client [nom=" + nom + ", prenom=" + prenom + ", email="
            + email + ", adressePostale=" + adressePostale
            + ", cp=" + cp + ", ville=" + ville + ", pays=" + pays
            + ", telephone=" + telephone + ", getId()=" + getId()
            + ", getPassword()=" + getPassword() + ", getPseudo()="
            + getPseudo() + ", getSecurityQuestion()=" + getSecurityQuestion() + ", getAnswer()="
            + getAnswer() + ", getRoles()=" + getRoles() + ", getClass()="
            + getClass() + ", hashCode()=" + hashCode() + ",
            toString()=" + super.toString() + "]";
    }
}
```

Les classes DAO (Data Access Object)

Les classes DAO sont responsables de l'accès aux données (lecture, création, mise à jour, suppression) dans le système de persistance (généralement une base de données). Elles interagissent avec les classes du modèle pour transférer les instances d'objets entre la base de données et le modèle. Le DAO est un service orienté CRUD (Create, Read, Update, Delete) pour les données. Elles contiennent les méthodes utilisées dans les contrôleurs.

```
package com.nanasenseimvc.dao;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

import com.nanasenseimvc.model.User;

// Data Access Object (DAO) class for User operations
public class UserDao {
    private Connection con;
    private String query;
    private PreparedStatement pst;
    private ResultSet rs;
    private Statement st;

    // Constructor that takes a database connection
    public UserDao(Connection con) {
        this.con = con;
    }

    // Method to authenticate user login
    public User userLogin(String email, String password) {
        User user = null;
        try {
            // SQL query to fetch user details based on email and password
            query = "select * from users where email=? and motDePasse=?";
            // Use prepared statement to prevent SQL injection
            pst = this.con.prepareStatement(query);
            pst.setString(1, email);
            pst.setString(2, password);
            rs = pst.executeQuery(); // Execute the query and get results
            if(rs.next()) { // If a matching user is found
                user = new User();
                user.setId(rs.getInt("idUser"));
                user.setPseudo(rs.getString("pseudo"));
                user.setEmail(rs.getString("email"));
                user.setRoles(rs.getString("roles"));
            }
        } catch (SQLException e) {
            System.out.println(e.getMessage());
        }
        return user;
    }

    // Method to handle forgot password functionality
    public User userForgotPassword(String email, String securityQuestion,
        String answer, String userPassword) {
        int check = 0;
        User user = null;
        try {
            // SQL query to check user's security question and answer
            query = "SELECT * FROM users WHERE email='" + email + "' and questionDeSecuritee='" + securityQuestion + "' and reponse='" + answer + "'";
            st = this.con.createStatement();
            rs = st.executeQuery(query);
            while(rs.next()) {
                check = 1;
            }
            // Update password if security check passes
            st.executeUpdate("update users set motDePasse='" + userPassword + "' where email='" + email + "'");
            System.out.println("Password Updated !");
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
        return user;
    }

    // Method to delete a user
    public User deleteUser(int id) {
        User user = null;
        try {
            // SQL query to delete a user
            query = "DELETE from users WHERE idUser=?";
            pst = this.con.prepareStatement(query);
            pst.setInt(1, id);
            pst.executeUpdate();
        } catch (Exception e) {
            // Exception handling (currently empty)
        }
        return user;
    }
}

package com.nanasenseimvc.dao;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;

import com.nanasenseimvc.model.Client;

// Data Access Object (DAO) class for Client operations
public class ClientDao {
    private Connection con;
    private String query;
    private PreparedStatement pst;

    // Constructor that takes a database connection
    public ClientDao(Connection con) {
        super();
        this.con = con;
    }

    // Method to update or insert client data
    public Client clientDataUpdate(int id, String nom, String prenom,
        String email, String adressePostale,
        String cp, String ville, String pays, String telephone) {
        Client client = null;
        try {
            // SQL query to insert client data
            query = "INSERT INTO client (idUser,nomClient, prenomClient,mailClient,adressePostaleClient, codePostaleClient, villeClient, paysClient,numTelClient) values (?, ?, ?, ?, ?, ?, ?, ?, ?)";
            pst = this.con.prepareStatement(query);
            // Set parameters for the prepared statement
            pst.setInt(1, id);
            pst.setString(2, nom);
            pst.setString(3, prenom);
            pst.setString(4, email);
            pst.setString(5, adressePostale);
            pst.setString(6, cp);
            pst.setString(7, ville);
            pst.setString(8, pays);
            pst.setString(9, telephone);
            // Execute the update
            pst.executeUpdate();
        } catch (SQLException e) {
            System.out.println(e.getMessage());
        }
        return client;
    }

    // Method to delete a client
    public Client deleteClient(int id) {
        Client client = null;
        try {
            // SQL query to delete a client
            query = "DELETE from client WHERE idUser=?";
            pst = this.con.prepareStatement(query);
            // Set the id parameter
            pst.setInt(1, id);
            // Execute the delete operation
            pst.execute();
        } catch (Exception e) {
            // Exception handling (currently empty)
        }
        return client;
    }
}
```


Les vues

La création d'un compte

Créer son compte pour interagir avec le site

Votre adresse mail
ADMIN@XPROD.FR

Votre pseudo
VOTRE PSEUDO

Votre mot de passe

La question de sécurité
QUEL EST TON PREMIER MANGA ▼

Votre réponse
VOTRE RÉPONSE

CRÉER LE COMPTE

DÉJÀ UN COMPTE ? CONNECTE-TOI ICI !

Newsletter
Adresse email
ADRESSE EMAIL
Souscrire

Sur cette première page liée à la connexion, nous retrouvons :

- L'importation du head.
- L'importation du CSS spécifique à la connexion
- Un formulaire qui a pour action « signup » qui est le nom de la servlet permettant d'interagir avec les modèles concernant les informations de connexion que nous verrons par la suite.
- Un lien vers la page de connexion si le compte existe déjà.
- L'importation du footer.

```
<? page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html lang="fr">
<head>
<!-- Set character encoding and viewport for responsive design -->
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Nana-sensei - Connexion</title>

<!-- Include common head elements -->
<%include file="includes/head.jsp" %>
<!-- Include CSS styles for this page -->
<style>
<%include file="CSS/connexion.css"%>
</style>
</head>
<body>
<!-- Include navigation menu -->
<%include file="includes/menu.jsp" %>
<main role="main">
<!-- Sign-up section with ARIA attributes for accessibility -->
<section role="region" class="signup" aria-labelledby="createaccount">
<div class="container">
<h2 id="createaccount">Créer son compte pour interagir avec le site</h2>
<!-- Sign-up form -->
<form action="signup" method="post" class="account">
<div class="line1">
<label for="email">Votre adresse mail</label> <br>
<input type="email" name="email" class="input" placeholder="Votre adresse mail"> <br>
</div>
<div class="line2">
<label for="pseudo">Votre pseudo</label> <br>
<input type="text" name="pseudo" class="input" placeholder="Votre pseudo"> <br>
</div>
<div class="line3">
<label for="password">Votre mot de passe</label> <br>
<input type="password" name="password" class="input" placeholder="Mot de passe"> <br>
</div>
</div>
<div class="line4">
<label for="securityQuestion">La question de sécurité</label> <br>
<!-- Dropdown for security question selection -->
<select name="securityQuestion" id="securityQuestion">
<option value="manga">Quel est ton premier manga ?</option>
<option value="dessin">Quelle est ta couleur préférée?</option>
<option value="animaux">Comment s'appelle ton premier animal de compagnie ?</option>
</select> <br>
</div>
<div class="line5">
<label for="answer">Votre réponse</label> <br>
<input type="text" name="answer" class="input" placeholder="Votre réponse"> <br>
</div>
<div class="center">
<button class="ctalogin" type="submit">Créer le compte</button>
</div>
<!-- Link to login page for existing users -->
<p class="optionco">Déjà un compte ?<a class="optionco" href="login.jsp">Connecte toi ici !</a></p>
</div>
</section>
</main>
<!-- Include footer -->
<%include file="includes/footer.jsp" %>
</body>
</html>
```

La connexion à son compte

Nana-sensei

Cours Boutique Blog Connexion

Se connecter

Adresse mail
ADMIN@XPROD.FR

Mot de passe

SE CONNECTER

MOT DE PASSE OUBLIÉ ? C'EST ICI !
PAS ENCORE DE COMPTE ? REJOINS LA COMMUNAUTÉ ICI !

Nana-sensei

Cours Boutique Blog Contact

Newsletter
Adresse email
ABONNÉ
Souscrire

© 2024 NANA-SENSEI | TOUS DROITS RÉSERVÉS | MENTIONS LÉGALES | CONDITIONS GÉNÉRALES DE VENTE | POLITIQUE DE CONFIDENTIALITÉ

Sur la page de connexion, nous retrouvons :

- L'importation du head.
- L'importation du CSS spécifique à la connexion
- Un formulaire qui a pour action « userlogin » qui est le nom de la servlet permettant d'interagir avec les modèles concernant les informations de connexion que nous verrons par la suite.
- Un lien vers la page de création de compte si celui-ci n'existe pas et un vers la page permettant de créer un nouveau mot de passe en cas d'oubli.
- L'importation du footer.

```
login.jsp
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html lang="fr">
<head>
<!-- Set the character encoding and viewport for responsive design -->
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Nana-sensei - Connexion</title>
<!-- Include the head.jsp file which likely contains common head elements -->
<%@include file="includes/head.jsp" %>
<!-- Include the CSS file for styling -->
<style>
    <@include file="CSS/connexion.css">
</style>
</head>
<body>
<!-- Include the menu.jsp file which likely contains the navigation menu -->
<%@include file="includes/menu.jsp" %>
<main>
    <section class="login">
        <div class="container">
            <h1>Se connecter</h1>
            <!-- Login form -->
            <form action="userlogin" method="post">
                <div class="line">
                    <!-- Email input field -->
                    <label class="labels" for="login-email">Adresse mail</label>
                    <input type="email" id="login-email" name="login-email"
                        class="inputco" required aria-required="true">
                </div>
                <div class="line">
                    <!-- Password input field -->
                    <label class="labels" for="login-password">Mot de passe</label>
                    <input type="password" id="login-password" name="login-password"
                        class="inputco" required aria-required="true">
                </div>
                <!-- Submit button -->
                <button class="ctalogin" type="submit">Se connecter</button>
            </form>
            <!-- Link to forgot password page -->
            <p class="optionco">Mot de passe oublié ? <a href="forgotpassword.jsp">C'est
ici !</a></p>
            <!-- Link to sign up page -->
            <p class="optionco">Pas encore de compte ? <a href="signup.jsp">Rejoins la
communauté ici !</a></p>
        </div>
    </section>
</main>
<!-- Include the footer.jsp file -->
<%@include file="includes/footer.jsp" %>
</body>
</html>
```


En cas d'oubli du mot de passe

Mot de passe oublié

Votre adresse mail
ADMIN@XPROD.FR

Votre nouveau mot de passe

La question de sécurité
QUEL EST TON PREMIER MANGA ?

Votre réponse
VOTRE RÉPONSE

MODIFIER LE MOT DE PASSE

Sur cette page permettant de redéfinir un nouveau mot de passe en cas d'oubli, nous retrouvons :

- L'importation du head.
- L'importation du CSS spécifique à la connexion
- Un formulaire qui a pour action « forgotpassword » qui est le nom de la servlet permettant d'interagir avec les modèles concernant les informations de connexion que nous verrons par la suite.
- Un lien vers la page de création de compte si celui-ci n'existe pas et un vers la page permettant de créer un nouveau mot de passe en cas d'oubli.
- L'importation du footer.

```
for forgotpassword.jsp

<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<!-- Set character encoding -->
<meta charset="UTF-8" />
<title>Manga-sensei - Mot de passe oublié</title>
<!-- Include common head elements -->
<!-- Include CSS styles for this page -->
<style>
<!-- Include file="CSS/connexion.css"%>
</style>
</head>
<body>
<!-- Include navigation menu -->
<!-- Include file="includes/menu.jsp" %>
<main role="banner">
<!-- Forgot password section with ARIA attributes for accessibility -->
<section role="region" aria-labelledby="forgotpassword">
<div class="container">
<div id="forgotpassword">
<h1 id="forgotpassword">Mot de passe oublié</h1>
<!-- Forgot password form -->
<form action="forgotpassword" method="post">
<div class="line1">
<label for="email">Votre adresse mail</label> <br>
<input type="email" name="email" class="input" placeholder="Votre adresse mail" required> <br>
</div>
<div class="line2">
<label for="password">Votre nouveau mot de passe</label> <br>
<input type="password" name="newPassword" class="input" placeholder="Nouveau mot de passe" required> <br>
</div>
</div>
</div>
</main>
<!-- Include footer.jsp file -->
<!-- Include file="includes/footer.jsp" %>
</body>
</html>
```

Un accès pour modifier ses données



Cours

Boutique

Blog

Hello 

Mes données personnelles

Modifier le mot de passe

Nouveau mot de passe

NOUVEAU MOT DE PASSE

Confirmation du mot de passe

CONFIRMATION

MODIFIER

Informations de facturation

Votre nom

VOTRE NOM

Votre prénom

VOTRE PRENOM

Votre mail

CATHY@CATHY.FR

Votre adresse postale

VOTRE ADRESSE POSTALE

Votre code postal

CODE POSTAL

Votre ville

VOTRE VILLE

Votre pays

VOTRE PAYS

Votre numéro de téléphone

NUMÉRO DE TÉLÉPHONE

MODIFIER

Supprimer mon compte

Mes données personnelles

Mes articles favoris

Mes commandes

Mes cours



Cours

Boutique

Blog

Contact



Newsletter

Adresse email

ADRESSE EMAIL

Souscrire

© 2024 NANA-SENSEI | TOUS DROITS RÉSERVÉS | MENTIONS LÉGALES | CONDITIONS GÉNÉRALES DE VENTE | POLITIQUE DE CONFIDENTIALITÉ

Par Cathy NAVARRO 27

L'utilisateur a possibilité une fois connecté de modifier son mot de passe et son adresse dans la section « Données personnelles » en cliquant sur son nom en haut à droite.

```
account.jsp

<%@page import="com.nanasenseimvc.controller.ClientUpdateDataServlet"%>
<%@page import="com.nanasenseimvc.dao.ClientDao"%>
<%@page import="com.nanasenseimvc.model.Client"%>
<% page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Nana-sensei - Mon compte</title>
<!-- Include common head elements -->
<%include file="includes/head.jsp" %>
<!-- Include CSS styles -->
<style>
<%include file="CSS/connexion.css"%>
<%include file="CSS/menuLog.css"%>
</style>
</head>
<body>
<!-- Include navigation menu -->
<%include file="includes/menu.jsp" %>
<main role="main">
<!-- Side navigation menu for account-related pages -->
<div class="menuLog">
<ul>
<li><a href="account.jsp" class="account"> Mes données personnelles</a></li>
<li><a href="mesfavoris.jsp" class="favoris"> Mes articles favoris</a></li>
<li><a href="mescommandes.jsp" class="commande"> Mes commandes</a></li>
<li><a href="mescours.jsp" class="mescours"> Mes cours </a></li>
</ul>
</div>
<section role="region" class="container61" aria-labelledby="donneespersonnelle">
<article>
<h1 id="donneespersonnelle">Mes données personnelles</h1>
<div class="mdpchange">
<h2>Modifier le mot de passe</h2>
<!-- Form for changing password -->
<form action="forgetpassword" method="post">
<input type="hidden" name="csrfToken" value="${sessionScope.csrfToken}"/>
<label class="label" for="newpassword">Nouveau mot de passe</label><br>
<input type="password" name="newpassword" placeholder="Nouveau mot de passe"><br>
<label class="label" for="confirmation">Confirmation du mot de passe</label><br>
<input type="password" name="confirmation" placeholder="Confirmation"><br>
<button type="submit" class="ctaLogin">Modifier</button>
</form>
</div>
</article>
</article>
```

```
account.jsp

<h2>Informations de facturation</h2>
<!-- Check if client data is available -->
<% Client clientData = (Client)request.getAttribute("clientData");
if(clientData != null) { %>
<!-- Form for updating client information -->
<form action="clientupdatedata" method="post">
<!-- Form fields for client data (pre filled)-->
<div class="line1">
<input type="hidden" name="id" value="${auth.getId()} %>
<label class="label" for="nom">Votre nom</label><br>
<input type="text" name="nom" value="${ clientData.getNom() %>
placeholder="Votre nom" required><br>
</div>
<div class="line2">
<label class="label" for="prenom">Votre prénom</label><br>
<input type="text" name="prenom" value="${ clientData.getPrenom() %>
placeholder="Votre prenom" required><br>
</div>
<div class="line3">
<label class="label" for="mail">Votre mail</label><br>
<input type="email" name="mail" value="${ auth.getEmail() %> placeholder="Votre mail" required><br>
</div>
<div class="line4">
<label class="label" for="adresse">Votre adresse postale</label><br>
<input type="text" name="adresse" value="${ clientData.getAdressePostale() %>
placeholder="Votre adresse postale" required><br>
</div>
<div class="line5">
<label class="label" for="cp">Votre code postal</label><br>
<input type="text" name="cp" value="${ clientData.getCp() %> placeholder="Code postal" required><br>
</div>
<div class="line6">
<label class="label" for="ville">Votre ville</label><br>
<input type="text" name="ville" value="${ clientData.getVille() %>
placeholder="Votre ville" required><br>
</div>
<div class="line7">
<label class="label" for="pays">Votre pays</label><br>
<input type="text" name="pays" value="${ clientData.getPays() %>
placeholder="Votre pays"><br>
</div>
<div class="line8">
<label class="label" for="tel">Votre numéro de téléphone</label><br>
<input type="tel" name="tel" value="${ clientData.getTelephone() %>
placeholder="numéro de téléphone" required><br>
</div>
<button type="submit" class="ctaLogin">Modifier</button>
</form>
```

```
account.jsp

<% } else { %>
<form class="formUp" action="clientupdatedata" method="post">
<!-- Form for updating client information (empty fields) -->
<input type="hidden" name="id" value="${auth.getId()} %>
<div class="line1">
<label class="label" for="nom">Votre nom</label><br>
<input type="text" name="nom" value="" placeholder="Votre nom" required><br>
</div>
<div class="line2">
<label class="label" for="prenom">Votre prénom</label><br>
<input type="text" name="prenom" value="" placeholder="Votre prenom" required><br>
</div>
<div class="line3">
<label class="label" for="mail">Votre mail</label><br>
<input type="email" name="mail" value="" auth.getEmail() %> placeholder="Votre mail" required><br>
</div>
<div class="line4">
<label class="label" for="adresse">Votre adresse postale</label><br>
<input type="text" name="adresse" value="" placeholder="Votre adresse postale" required><br>
</div>
<div class="line5">
<label class="label" for="cp">Votre code postal</label><br>
<input type="text" name="cp" value="" placeholder="Code postal" required><br>
</div>
<div class="line6">
<label class="label" for="ville">Votre ville</label><br>
<input type="text" name="ville" value="" placeholder="Votre ville" required><br>
</div>
<div class="line7">
<label class="label" for="pays">Votre pays</label><br>
<input type="text" name="pays" value="" placeholder="Votre pays"><br>
</div>
<div class="line8">
<label class="label" for="tel">Votre numéro de téléphone</label><br>
<input type="tel" name="tel" value="" placeholder="numéro de téléphone" required>
<br>
</div>
<button type="submit" class="ctaLogin">Modifier</button>
</form>
<% } %>
</article>
<!-- Link to delete account -->
<a type="submit" class="supress" href="deleteaccount?id=${auth.getId()} %>Supprimer mon compte</a>
</section>
</main>
<!-- Include footer -->
<%include file="includes/footer.jsp" %>
</body>
</html>
```

L'administrateur peut voir la liste des utilisateurs inscrit sur son site

Cours

Boutique

Blog

Connexion

Gestion des utilisateurs

Gestion des articles

Gestion de la boutique

Gestion des cours

Gestion des utilisateur

Pseudo	Email	Nom	Prénom	Adresse postale	Code postale	Ville	Pays	téléphone
KYUZY	ADMIN@XPROD.FR	BASSINGHA	SEPTIME	3 RUE PIERRE CURLOT	75100	EVRY	FRANCE	061257215
NANASENSEI	CONTACT@NANA-SENSEI.FR	BASSINGHA	SEPTIME	3 RUE PIERRE CURLOT	75100	EVRY	FRANCE	061523456484
HGDBKSL	CATHY@IRFA.FR	NULL	NULL	NULL	NULL	NULL	NULL	NULL
FREDDY	FRED@IRFA.FR	FRED	FRED	1 RUE DE L'IRFA	71290	CREUSOT	FRANCE	0678945134
HELLO	CATHY@CATHY.FR	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Cours

Boutique

Blog

Contact

Menu admin

© 2024 NANA-SENSEI | TOUS DROITS RÉSERVÉS | Mentions légales | Conditions générales de vente | Politique de confidentialité

Grâce à une vue de la base de données, l'administrateur à une visibilité sur les informations des utilisateurs.

Celles-ci apparaissent dans un tableau.

adminUser.jsp

```
<@page import="java.sql.Statement"%>
<@page import="java.sql.ResultSet"%>
<@page import="com.nanasensei.mvc.connection.DbCon"%>
<@page import="java.sql.Connection"%>
<% page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8"%>
<title>Nana-sensei | User Management</title>

<!-- Include common head elements -->
<%include file="../includes/head.jsp"%>
<!-- Include CSS styles for this page -->
<style>
<%include file="../CSS/adminUserCss.css"%>
<%include file="../CSS/menuLog.css"%>
</style>
</head>
<body>
<!-- Include admin menu -->
<%include file="adminmenu.jsp"%>
<main role="banner">
<div class="menuLog">
<ul>
<li class="account"><a href="adminUser.jsp" class="account"> User Management</a>
</li>
<li>
<li><a href="adminBlog.jsp" class="favorite"> Article Management</li>
<li><a href="adminboutique.jsp" class="command">Shop Management</li>
<li><a href="adminCours.jsp" class="mescours"> Course Management </li>
</li>
</div>
<!-- Main container for user management -->
<div class="container">
<h1>User Management</h1>
<!-- Table to display user information -->
<table>
<thead>
<tr>
<th scope="col">Username</th>
<th scope="col">Email</th>
<th scope="col">Last Name</th>
<th scope="col">First Name</th>
<th scope="col">Postal Address</th>
<th scope="col">Postal Code</th>
<th scope="col">City</th>
<th scope="col">Country</th>
<th scope="col">Phone Number</th>
</tr>
</thead>
<tbody>
```

adminUser.jsp

```
<!-- Fetch and display user data from the database -->
<%
try {
// Establish a connection to the database
Connection con = DbCon.getConnection();
Statement st = con.createStatement();
// Execute a query to retrieve user data from the view
ResultSet rs = st.executeQuery("SELECT * from view_usersadmin");
while (rs.next()) {
%>
<!-- Populate table rows with user data -->
<tr>
<td scope="col"><%= rs.getString(1) %></td> <!-- Username -->
<td scope="col"><%= rs.getString(2) %></td> <!-- Email -->
<td scope="col"><%= rs.getString(3) %></td> <!-- Last Name -->
<td scope="col"><%= rs.getString(4) %></td> <!-- First Name -->
<td scope="col"><%= rs.getString(5) %></td> <!-- Postal Address -->
<td scope="col"><%= rs.getString(6) %></td> <!-- Postal Code -->
<td scope="col"><%= rs.getString(7) %></td> <!-- City -->
<td scope="col"><%= rs.getString(8) %></td> <!-- Country -->
<td scope="col"><%= rs.getString(9) %></td> <!-- Phone Number -->
</tr>
<%
}
} catch (Exception e) {
// Handle exceptions and print error messages
System.out.println(e.getMessage());
}
%>
</tbody>
</table>
</div>
</main>
<!-- Include footer -->
<%include file="includes/footer.jsp"%>
</body>
</html>
```

Un fichier CSS commun à toute les pages liés à la connexion...

```
connexion.css

.container6{
  display: flex;
  flex-direction: column;
  flex-wrap: nowrap;
  align-items: center;
  margin : 2rem 2rem 2rem 2rem;
  padding: 2rem 2rem 2rem 2rem;
  background-color: var(--lightconnexion);
  border-radius: 50px;
}
h1{font-family: 'Cherry Bomb One';
color: var(--blackconnexion);
letter-spacing: 2px;
margin-bottom: 2rem;
font-size: 30px;}
h2{
  font-family: 'Cherry Bomb One';
  color: var(--blackconnexion);
  letter-spacing: 2px;
  margin-bottom: 1rem;
  font-size: 20px;
  text-align:center;
}
.line{
  display:flex;
  flex-direction:column;
  align-items: flex-start;
}
.labels{
  font-family: 'Cherry Bomb One';
  color: var(--blackconnexion);
  letter-spacing: 1px;
  margin-top: 1rem;
}
.inputco{
  border: 2px solid var(--flashconnexion);
  border-radius: 50px;
  padding: 0.5rem 0.5rem 0.5rem 0.5rem;
  background-color: var(--lightconnexion);
  margin-bottom: 1rem;
}
input {
  border: 2px solid var(--flashconnexion);
  border-radius: 50px;
  padding: 0.5rem 0.5rem 0.5rem 0.5rem;
  background-color: var(--lightconnexion);
  margin-bottom: 1rem;
  color: var(--blackconnexion);
  font-family: 'Sailors';
  padding-left: 1rem;
}
::placeholder{
  color: var(--blackconnexion);
  font-family: 'Sailors';
  padding-left: 1rem;
}
}
.ctalogin{
border: 3px solid var(--flashconnexion);
border-radius: 50px;
background-color: var(--lightconnexion);
font-family: 'Sailors';
color: var(--blackconnexion);
letter-spacing: 1px;
padding: 0.25rem 1rem 0.25rem 1rem;
margin:1rem;
width: fit-content;
}
.ctalogin:hover{
border-radius: 50px;
background-color: var(--blackconnexion);
color: white;
}
.optionco{
font-family: 'Sailors';
font-size: 12px;
font-weight: bold;
color: var(--blackconnexion);
}
```

```
connexion.css

.formUp{
  display: grid;
  column-gap: 1rem;
}
.line1{
  grid-row: 1;
  grid-column: 1;
}
.line2 {
  grid-row: 1;
  grid-column: 3;
}
.line3 {
  grid-row: 2;
  grid-column: 1;
}
.line4 {
  grid-row: 2;
  grid-column: 3;
}
.line5 {
  grid-row: 3;
  grid-column: 1;
}
.line6 {
  grid-row: 3;
  grid-column: 3;
}
.line7 {
  grid-row: 4;
  grid-column: 1;
}
.line8 {
  grid-row: 4;
  grid-column: 3;
}
.line9 {
  grid-row: 5;
  grid-column: 1;
}
.line10 {
  grid-row: 5;
  grid-column: 3;
}
.line11 {
  grid-row: 6;
  grid-column: 1;
}
```

```
connexion.css

.ctaloginup{
  grid-row : 7;
  grid-column : 2;
}
#securityQuestion{
  width:212.8px;
  height:43.8px;
  color: var(--blackconnexion);
  font-family: 'Sailors';
  border: 2px solid var(--flashconnexion);
  border-radius: 50px;
  background-color: var(--lightconnexion);
  margin-bottom: 1rem;
}
.center{
  grid-row: 4;
  grid-column: 1 /span 3;
}
.container61{
  width:80%;
  display: flex;
  flex-direction: column;
  flex-wrap: nowrap;
  align-items: center;
  margin: 2rem 2rem 2rem 2rem;
  padding: 2rem 2rem 2rem 2rem;
  background-color: var(--lightconnexion);
  border-radius: 50px;
}
.mdpchange{
margin-bottom: 3rem;
}
.container6 form{
  text-align: center;
}
.infochange{
  display: grid;
  grid-template-columns: 3fr 3fr 3fr;
  align-items: center;
  justify-items: center;
  align-content: center;
  grid-row-gap: 1rem;
}
.supress{
  background-color : red;
  color:white;
  font-family: 'Cherry Bomb One';
  border-radius: 50px;
  border: 3px solid;
  letter-spacing: 1px;
  padding: 0.25rem 1rem 0.25rem 1rem;
  margin-top:2rem;
  text-decoration:none;
}
```

...et un CSS pour la partie administrateur

```
adminuser.css

.container6{
  display: flex;
  flex-direction: column;
  flex-wrap: nowrap;
  align-items: center;
  margin : 2rem 2rem 2rem 2rem;
  padding: 2rem 2rem 2rem 2rem;
  background-color: var(--lightconnexion);
  border-radius: 50px;
}
li.account{
border : 3px solid var(--flashconnexion);}
h1 {
  font-family: 'Cherry Bomb One';
  color: var(--blackconnexion);
  letter-spacing: 2px;
  margin-bottom: 2rem;
  font-size: 30px;}
```

```
adminuser.css

table{
border-radius: 50px;
border: 2px solid var(--blackconnexion);
padding: 1rem 1rem 1rem 1rem;}
th{
padding: 0.5rem;
border-bottom: 2px solid var(--blackconnexion);
font-family: 'Cherry Bomb One';
color: var(--blackconnexion);
letter-spacing: 2px;
margin-bottom: 2rem;
margin-right: 1rem;}
td{
padding: 0.5rem;
border-bottom: 1px solid var(--blackconnexion);
font-family: 'Sailors';
color: var(--blackconnexion);
letter-spacing: 2px;
margin-bottom: 2rem;
margin-right: 1rem;}
```

Les contrôleurs

Les servlets sont des programmes Java qui tournent sur un serveur web pour traiter les requêtes des clients et générer des réponses dynamiques, formant ainsi la base de nombreuses applications web Java.

La servlet qui permet la création d'un compte

```
package com.nanasenseimvc.controller;

// Import necessary Java and Servlet libraries
import java.io.IOException;
import java.sql.Connection;
import java.sql.PreparedStatement;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

// Import custom classes for database connection and encryption
import com.nanasenseimvc.connection.DbCon;
import com.nanasenseimvc.model.Encryption;

// Servlet mapping for the signup process
@WebServlet ("/signup")
public class SignUpServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    // Constructor
    public SignUpServlet() {
        super();
    }
    // Handle GET requests (redirect to login page)
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        response.sendRedirect("login.jsp");
    }
    // Handle POST requests (process signup form)
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        response.setContentType("text/html; charset=UTF-8");
        try{
            // Set default role as "client"
            String roles= "client";
            // Retrieve form data
            String email = request.getParameter("email");
            String pseudo = request.getParameter("pseudo");
            String password = request.getParameter("password");
            String securityQuestion = request.getParameter("securityQuestion");
            String answer = request.getParameter("answer");
            // Encrypt the password
            String userPassword=Encryption.getSHA1(password);
            // Establish database connection
            Connection con = DbCon.getConnection();
            // Prepare SQL statement for inserting new user
            PreparedStatement pst = con.prepareStatement("INSERT INTO users(roles, email, pseudo, motDePasse, questionDeSecurite, reponse) values ( ?, ?, ?, ?, ?, ? );");
            // Set parameters for the prepared statement
            pst.setString(1, roles);
            pst.setString(2, email);
            pst.setString(3, pseudo);
            pst.setString(4, userPassword);
            pst.setString(5, securityQuestion);
            pst.setString(6, answer);
            // Execute the insert statement
            pst.executeUpdate();
            // Redirect to login page after successful signup
            response.sendRedirect("login.jsp");
        }
        catch(Exception e)
        {
            // Handle exceptions
            e.printStackTrace();
            e.getMessage();
            System.out.print("No user register!");
            // Redirect back to signup page with error message
            response.sendRedirect("signup.jsp?msg=invalid");
        }
    }
}
```

Cette servlet permet la création d'un compte utilisateur.

L'utilisateur remplit les informations du formulaire d'action « signup ». Celles-ci sont récupérées par la servlet qui après s'être connectée à la base de données, prépare les informations dans une requête (pour assurer la sécurité) et les envoie à la base de donnée. Si l'utilisateur est correctement enregistré, il est redirigé sur la page de connexion.

La servlet qui permet la connexion

Cette servlet vérifie que l'utilisateur entre des informations dans le formulaire de connexion qui correspondent à ce qui se trouve dans la base de donnée et les réorienter vers l'espace qui est le sien. Elle utilise la méthode « userlogin() » présent dans le UserDao.

```
package com.nanasenseimvc.controller;

import java.io.IOException;
import java.io.PrintWriter;
import java.security.NoSuchAlgorithmException;
import java.sql.SQLException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

import com.nanasenseimvc.connection.DbCon;
import com.nanasenseimvc.dao.UserDao;
import com.nanasenseimvc.model.Encryption;
import com.nanasenseimvc.model.User;

// Servlet mapping for user login
@WebServlet("/userLogin")
public class LoginServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    // Constructor
    public LoginServlet() {
        super();
    }

    // Handle GET requests (redirect to login page)
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        response.sendRedirect("login.jsp");
    }

    // Handle POST requests (process login form)
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        response.setContentType("text/html; charset=UTF-8");
        HttpSession session = request.getSession();
        try (PrintWriter out = response.getWriter()) {
            // Retrieve login form data
            String email = request.getParameter("login-email");
            String password = request.getParameter("login-password");
            // Encrypt the password
            String userPassword = Encryption.getSHA1(password);

            // Authenticate user
            UserDao userDao = new UserDao(DbCon.getConnection());
            User user = userDao.userLogin(email, userPassword);

            if (user != null) {
                if (user.getRoles().equals("admin")) {
                    // Admin login
                    request.getSession().setAttribute("auth", user);
                    session.setAttribute("email", email);

                    System.out.print(" ADMIN Login: ");
                    System.out.print(" Admin: " + email + " logged in");
                    response.sendRedirect("admin/adminHome.jsp");
                } else {
                    // Client login
                    request.getSession().setAttribute("auth", user);
                    response.sendRedirect("index.jsp");

                    System.out.print("CLIENT login");
                    System.out.print(" Client: " + email + " logged in");
                }
            } else {
                // Login failed
                response.sendRedirect("index.jsp?msg=WrongPasswordOrEmail");
                System.out.print("Wrong password or email");
            }
        } catch (ClassNotFoundException | SQLException | NoSuchAlgorithmException e) {
            e.printStackTrace();
        }
    }
}
```


La servlet qui permet de modifier le mot de passe

Cette servlet est appelée dans le formulaire d'oubli de mot de passe ainsi que dans la modification du mot de passe dans l'espace « données personnelles » du compte.

Elle récupère via un formulaire les données rentrées par l'utilisateur, elle hache le mot de passe, se connecte à la base de donnée et utilise la méthode « userForgotPassword() » déclarée dans UserDao qui vérifie si les informations collectées et celles de la base de donnée correspondent. Si oui, le mot de passe est mis à jour et l'utilisateur est renvoyé sur la page « login.jsp ».

```
ForgotPasswordServlet.java

package oom.nanasenseimvc.controller;

import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import oom.nanasenseimvc.connection.DbCon;
import oom.nanasenseimvc.dao.UserDao;
import oom.nanasenseimvc.model.Encryption;
import oom.nanasenseimvc.model.User;

// This servlet is mapped to the "/forgotpassword" URL
@WebServlet("/forgotpassword")
public class ForgotPasswordServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    // Constructor
    public ForgotPasswordServlet() {
        super();
    }

    // This method handles GET requests
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        response.setContentType("text/html; charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            // Get parameters from the request
            String email = request.getParameter("email");
            String securityQuestion = request.getParameter("securityQuestion");
            String answer = request.getParameter("answer");
            String newPassword = request.getParameter("newPassword");

            // Encrypt the new password
            String userPassword = Encryption.getSHA1(newPassword);

            // Connect to the database
            UserDao udao = new UserDao(DbCon.getConnection());

            // Attempt to change the user's password
            User user = udao.userForgotPassword(email, securityQuestion, answer, userPassword);

            System.out.print("User : " + email + " password changed!");
            response.sendRedirect("login.jsp");
        } catch (Exception e) {
            System.out.print("No user registered!");
            response.sendRedirect("singup.jsp?msg=invalid");
        }
    }

    // This method handles POST requests by calling doGet
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        doGet(request, response);
    }
}
```


La servlet qui permet de modifier les informations personnelles

Cette servlet est appelé dans un formulaire dans la section « données personnelles ». La méthode « doGet » permet de récupérer les informations disponibles dans la base de donnée grâce à la méthode « ClientById() ». La méthode « doPost » utilise les informations complétées par l'utilisateur dans le formulaire pour mettre à jour la base de données grâce à la méthode « ClientUpdateData() ».

```
ClientUpdateDataServlet.java

package com.nanasenseimvc.controller;

import java.io.IOException;
import java.io.PrintWriter;
import java.sql.SQLException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.nanasenseimvc.connection.DbCon;
import com.nanasenseimvc.dao.ClientDao;
import com.nanasenseimvc.model.Client;
import com.nanasenseimvc.model.User;

// Servlet mapping for "/clientupdatedata"
@WebServlet("/clientupdatedata")
public class ClientUpdateDataServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    public ClientUpdateDataServlet() {
        super();
    }

    // Handle GET requests
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
    ServletException, IOException {
        response.setContentType("text/html ; charset=UTF-8");
        request.setCharacterEncoding("UTF-8");
        try {
            // Retrieve the ID of the connected user
            User auth = (User) request.getSession().getAttribute("auth");
            if (auth != null) {
                int userId = auth.getId();
                // Retrieve client data from the database
                ClientDao cdao = new ClientDao(DbCon.getConnection());
                Client client = cdao.getClientById(userId);
                if (client != null) {
                    // Store client data in the request
                    request.setAttribute("clientData", client);
                }
            }
            // Redirect to the form page
            request.getRequestDispatcher("account.jsp").forward(request, response);
        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
            response.sendRedirect("account.jsp?msg=error");
        }
    }

    // Handle POST requests
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
    ServletException, IOException {
        doGet(request, response);
        try (PrintWriter out = response.getWriter()) {
            // Retrieve form data
            int id = Integer.parseInt(request.getParameter("id"));
            String nom = request.getParameter("nom");
            String prenom = request.getParameter("prenom");
            String mail = request.getParameter("mail");
            String adressePostale = request.getParameter("adresse");
            String cp = request.getParameter("cp");
            String ville = request.getParameter("ville");
            String pays = request.getParameter("pays");
            String telephone = request.getParameter("tel");
            // Update client data in the database
            ClientDao cdao = new ClientDao(DbCon.getConnection());
            Client clients = cdao.clientDataUpdate(id, nom, prenom, mail, adressePostale, cp,
            ville, pays, telephone);
            // Redirect to account page with success message
            response.sendRedirect("account.jsp?msg=done");
        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
            System.out.println("Account not updated");
            // Redirect to account page with error message
            response.sendRedirect("account.jsp?msg=wrong");
        }
    }
}
```

La servlet qui permet de supprimer son compte sur le site

Cette servlet est appelée sur un bouton « supprimer mon compte » qui est de couleur rouge et qui contraste avec le reste du site. Elle fonctionne de la façon suivante :

- Elle récupère l'id du compte qui est connecté
- Si l'id n'est pas null, une méthode dans UserDao et dans ClientDao permet la suppression des tables « user » et « client ».
- Une fois supprimer, l'utilisateur est automatiquement déconnecté et renvoyé à la page d'accueil.

```
DeleteAccountServlet.java

package com.nanasenseimvc.controller;

import java.io.IOException;
import java.io.PrintWriter;
import java.sql.SQLException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.nanasenseimvc.connection.DbCon;
import com.nanasenseimvc.dao.ClientDao;
import com.nanasenseimvc.dao.UserDao;

// Servlet mapping for "/deleteaccount"
@WebServlet ("/deleteaccount")
public class DeleteAccountServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    public DeleteAccountServlet() {
        super();
    }

    // Handle GET requests
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
    ServletException, IOException {
        response.setContentType("text/html ; charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            // Get the user ID from the request parameters
            String id = request.getParameter("id");
            if (id != null) {
                // Delete user from the database
                UserDao userDao = new UserDao(DbCon.getConnection());
                userDao.deleteUser(Integer.parseInt(id));

                // Delete client from the database
                ClientDao clientDao = new ClientDao(DbCon.getConnection());
                clientDao.deleteClient(Integer.parseInt(id));

                System.out.println("User deleted");

                // Remove user authentication and redirect to home page
                request.getSession().removeAttribute("auth");
                response.sendRedirect("index.jsp");
            } else {
                System.out.println("User didn't delete");
                response.sendRedirect("account.jsp?msg=wrong");
            }
        } catch (SQLException | ClassNotFoundException e) {
            e.printStackTrace();
            System.out.println(e.getMessage());
        }
    }

    // Handle POST requests (currently just calls doGet)
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
    ServletException, IOException {
        doGet(request, response);
    }
}
```

La servlet qui permet la déconnexion

Cette servlet permet la déconnexion de l'utilisateur. Quand il est connecté, le bouton connexion prend le pseudo de l'utilisateur et à côté apparaît un bouton  qui est lié à la servlet via le « ».

```
LogoutServlet.java

package com.nanasenseimvc.controller;

import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

// This annotation maps the servlet to the URL "/logout"
@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {
    // Unique identifier for serialization
    private static final long serialVersionUID = 1L;
    // Constructor
    public LogoutServlet() {
        super();
    }
    // This method handles GET requests to the "/logout" URL
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
        try (PrintWriter out = response.getWriter()) {
            // Check if the user is authenticated (has an "auth" attribute in the session)
            if (request.getSession().getAttribute("auth") != null) {
                // If authenticated, remove the "auth" attribute from the session
                request.getSession().removeAttribute("auth");
                // Redirect the user to the login page
                response.sendRedirect("login.jsp");
            } else {
                // If not authenticated, redirect the user to the index page
                response.sendRedirect("index.jsp");
            }
        }
    }
}
```

La base de donnée

1 • SELECT * FROM nana_sensei.users;

	idUser	roles	email	pseudo	motDePasse	questionDeSecurite	reponse
1	admin		admin@xprod.fr	kyuzy	b47d926911d4e6b8201f801a151f5b82d513cf09	manga	Blackbutler
2	client		contact@nana-sensei.fr	nanasensei	49a5551af57e55ae484849565055515b50505...	dessin	blue
3	client		cathy@irfa.fr	hgdbkl	55c52a56d4857ca51555450af5449e535753504...	manga	Blackbutler
4	client		fred@irfa.fr	freddy	f55c51bc49d564856e4852555150adf54555757...	dessin	vert
5	client		cathy@cathy.fr	Helo	f7c3bc1d808e04732adf679965ccc34ca7ae3441	manga	Blackbutler

Capture d'écran de la base de donnée de la table user

```
1 • SELECT * FROM nana_sensei.client;
```

Result Grid

Filter Rows:

Edit

Export/Import

Wrap Cell Content: IA

	idUser	nomClient	prenomClient	mailClient	adressePostaleClient	codePostaleClient	villeClient	paysClient	numtelClient
1	bassingha	septime	contact@nana-sensei.fr	3 rue pierre curiot	75100	evry	france	061257215	
2	bassingha	septime	contact@nana-sensei.fr	3 rue pierre curiot	75100	evry	france	061523456484	
4	fred	fred	fred@irfa.fr	1 rue de l'irfa	71290	creusot	france	0678945134	
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	

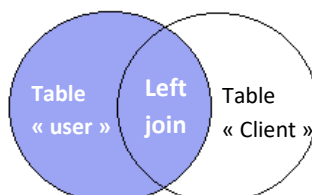
Capture d'écran de la base de donnée de la table client

1 • use nana_sensei;
2 • create view view_usersAdmin as
3 select pseudo, email, nomClient, prenomClient, adressePostaleClient, codePostaleClient, villeClient, paysClient, numTelClient
4 from nana_sensei.users left join nana_sensei.client on nana_sensei.users.idUser = nana_sensei.client.idUser;
5
6 • SELECT * FROM nana_sensei.view_usersadmin;

	pseudo	email	nomClient	prenomClient	adressePostaleClient	codePostaleClient	villeClient	paysClient	numTelClient
1	kyuzy	admin@xprod.fr	bassingha	septime	3 rue pierre curiot	75100	evry	france	061257215
2	nanasensei	contact@nana-sensei.fr	bassingha	septime	3 rue pierre curiot	75100	evry	france	061523456484
3	hgdbkl	cathy@irfa.fr							
4	freddy	fred@irfa.fr	fred	fred	1 rue de l'irfa	71290	creusot	france	0678945134
5	Helo	cathy@cathy.fr							

Capture d'écran de la base de donnée de la vue utilisée dans la page adminUser.jsp.

Cette dernière vue est obtenue en faisant une left join qui est une jointure externe pour retourner tous les enregistrements de la table de gauche (LEFT = gauche) même si la condition n'est pas vérifiée dans l'autre table. En d'autres termes, même si je n'ai pas de correspondance dans la table « client », les utilisateurs qui se sont créés un compte apparaîtront quand même dans ma vue.



e) La boutique
Les modèles

```
Product.java

package com.nanasenseimvc.model;

/**
 * Represents a product in the system.
 * This class encapsulates all the attributes of a product.
 */
public class Product {
    // Product attributes
    private int id;           // Unique identifier for the product
    private String name;      // Name of the product
    private String category;  // Category the product belongs to
    private double price;     // Price of the product
    private String image;     // URL or path to the product image
    private int stock;        // Current stock quantity
    private String tags;      // Tags associated with the product for search/filtering

    /**
     * Default constructor
     */
    public Product() {
        super();
    }

    /**
     * Parameterized constructor to initialize all product attributes
     */
    public Product(int id, String name, String category, double price, String image, int
stock, String tags) {
        super();
        this.id = id;
        this.name = name;
        this.category = category;
        this.price = price;
        this.image = image;
        this.stock = stock;
        this.tags = tags;
    }

    // Getter and Setter methods for all attributes
    public String getTags() {
        return tags;
    }
    public void setTags(String tags) {
        this.tags = tags;
    }
    public int getStock() {
        return stock;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getCategory() {
        return category;
    }
    public void setCategory(String category) {
        this.category = category;
    }
    public double getPrice() {
        return price;
    }
    public void setPrice(double price) {
        this.price = price;
    }
    public String getImage() {
        return image;
    }
    public void setImage(String image) {
        this.image = image;
    }
    public void setStock(int stock) {
        this.stock = stock;
    }
}
}
```

Ce sont les 2 modèles
utilisés dans la gestion de
la boutique.

```
Cart.java

// Package declaration
package com.nanasenseimvc.model;

// Cart class definition, extending the Product class
public class Cart extends Product {
    // Private field to store the quantity of the product in the cart
    private int quantity;

    // Getter method for the quantity
    public int getQuantity() {
        return quantity;
    }

    // Setter method for the quantity
    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }

    // Default constructor
    public Cart() {
        // Call the superclass (Product) constructor
        super();
    }
}
```

Les classes DAO (Data Access Object)

```
ProductDao.java

package com.nanasenseimvc.dao;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

import com.nanasenseimvc.model.Cart;
import com.nanasenseimvc.model.Product;

// Data Access Object for Product-related database operations
public class ProductDao {
    private Connection con;
    private String query;
    private PreparedStatement pst;
    private ResultSet rs;
    // Constructor initializing the database connection
    public ProductDao(Connection con) {
        super();
        this.con = con;
    }
    // Method to retrieve all products from the database
    public List<Product> getAllProducts() {
        List<Product> products = new ArrayList<Product>();
        try {
            // SQL query to select all products
            query = "select * from produit";
            // Prepare statement to prevent SQL injection
            pst = this.con.prepareStatement(query);
            // Execute query and get results
            rs = pst.executeQuery();
            // Iterate through results and create Product objects
            while (rs.next()) {
                Product row = new Product();
                // Set product attributes from database results
                row.setId(rs.getInt("idProduit"));
                row.setImage(rs.getString("photoProduit"));
                row.setName(rs.getString("nomProduit"));
                row.setCategory(rs.getString("descriptionProduit"));
                row.setPrice(rs.getDouble("prixUnitaireProduit"));
                row.setStock(rs.getInt("stock"));
                row.setTags(rs.getString("tags"));
                // Add product to the list
                products.add(row);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return products;
    }
    // Method to retrieve a single product by its ID
    public Product getSingleProduct(int id) {
        Product row = null;
        try {
            query = "SELECT * FROM produit WHERE idProduit=?";
            pst = this.con.prepareStatement(query);
            pst.setInt(1, id);
            rs = pst.executeQuery();

            while (rs.next()) {
                row = new Product();
                // Set product attributes from database results
                row.setId(rs.getInt("idProduit"));
                row.setName(rs.getString("nomProduit"));
                row.setCategory(rs.getString("descriptionProduit"));
                row.setPrice(rs.getDouble("prixUnitaireProduit"));
                row.setImage(rs.getString("photoProduit"));
            }
        } catch (Exception e) {
            e.printStackTrace();
            System.out.println(e.getMessage());
        }
        return row;
    }
}
```

Dans cette classe ProductDao se trouve le lien avec la base de données ainsi que les méthodes qui seront utilisées dans les contrôleurs de cette partie.

```
ProductDao.java

// Method to add a new product to the database
public Product addProduct(String reference, String image, String nom, String description,
double prix, int stock, String tags) {
    Product product = null;
    try {
        query = "insert into produit
(reference, photoProduit, nomProduit, descriptionProduit, prixUnitaireProduit, stock, tags) values
(?, ?, ?, ?, ?, ?, ?)";
        pst = this.con.prepareStatement(query);
        // Set parameters for the prepared statement
        pst.setString(1, reference);
        pst.setString(2, image);
        pst.setString(3, nom);
        pst.setString(4, description);
        pst.setDouble(5, prix);
        pst.setInt(6, stock);
        pst.setString(7, tags);
        pst.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
        System.out.println(e.getMessage());
    }
    return product;
}
// Method to delete a product from the database by its ID
public Product deleteProduct(int id) {
    Product product = null;
    try {
        query = "DELETE from produit WHERE idProduit=?";
        pst = this.con.prepareStatement(query);
        pst.setInt(1, id);
        pst.executeUpdate();
    } catch (Exception e) {
        // Exception handling is empty, consider adding proper error handling
    }
    return product;
}
// Method to retrieve cart products based on a list of Cart items
public List<Cart> getCartProducts(ArrayList<Cart> cartList) {
    List<Cart> products = new ArrayList<Cart>();
    try {
        // Check if there are items in the cart
        if (cartList.size() > 0) {
            // Iterate through each item in the cart
            for (Cart item : cartList) {
                query = "select * from produit where idProduit=?";
                pst = this.con.prepareStatement(query);
                pst.setInt(1, item.getId());
                rs = pst.executeQuery();
                // Create Cart objects from database results
                while (rs.next()) {
                    Cart row = new Cart();
                    row.setId(rs.getInt("idProduit"));
                    row.setName(rs.getString("nomProduit"));
                    row.setCategory(rs.getString("descriptionProduit"));
                    row.setPrice(rs.getDouble("prixUnitaireProduit") *
item.getQuantity());
                    row.setImage(rs.getString("photoProduit"));
                    row.setQuantity(item.getQuantity());
                    products.add(row);
                }
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
        System.out.println(e.getMessage());
    }
    return products;
}
}
```

Les vues

La boutique



Cette vue est la page d'accueil de la boutique, on peut y trouver :

- Les imports java,
- Les includes du head, du CSS spécifique à la boutique, du menu et du footer.
- Un formulaire de recherche utilisant la servlet « searchproduct »
- Un lien vers le panier
- Pour l'affichage de la page et des résultats de recherches, du Java a été utilisé. Pour commencer, les variables ont été déclarées et en fonction des résultats de la recherche, soit il n'y a pas de recherche et tous les produits sont affichés soit seuls les produits correspondant à la recherche sont affichés.

```

boutique.jsp

<%-- Import necessary Java classes --%>
<%@page import="java.util.ArrayList"%>
<%@page import="java.util.List"%>
<%@page import="com.nanasenseimvc.dao.ProductDao"%>
<%@page import="com.nanasenseimvc.model.Product"%>
<%@page import="java.sql.ResultSet"%>
<%@page import="java.sql.Statement"%>
<%@page import="com.nanasenseimvc.connection.DbCon"%>
<%@page import="java.sql.Connection"%>
<%-- Set the page language and character encoding --%>
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>

<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Nana-sensei | La boutique</title>
    <%-- Include the head.jsp file --%>
    <%@include file="includes/head.jsp" %>
    <%-- Include the CSS file for the boutique page --%>
    <style type="text/css">
    <%@include file="CSS/boutique.css"%>
    </style>
</head>
<body>
    <%-- Include the menu.jsp file --%>
    <%@include file="includes/menu.jsp" %>
    <h1 id="boutique">Bienvenue sur la boutique de Nana-sensei !</h1>

    <%-- Search form and shopping cart link --%>
    <div class="search-container">
        <div class="barresearch">
            <form action="searchproduct" method="post">
                <input id="recherche" type="text" placeholder="Rechercher un article"
name="search">
                <button id="recherche" type="submit"></button>
            </form>
            <div><a href="panier.jsp">
                
            </a></div>
        </div>
    </div>

    <section role="region">
        <article role="article">
            <%
// Initialize variables for product display
List<Product> productsToDisplay = new ArrayList<>();
String searchTerm = (String) request.getAttribute("searchTerm");
List<Product> searchResults = (List<Product>) request.getAttribute("searchResults");

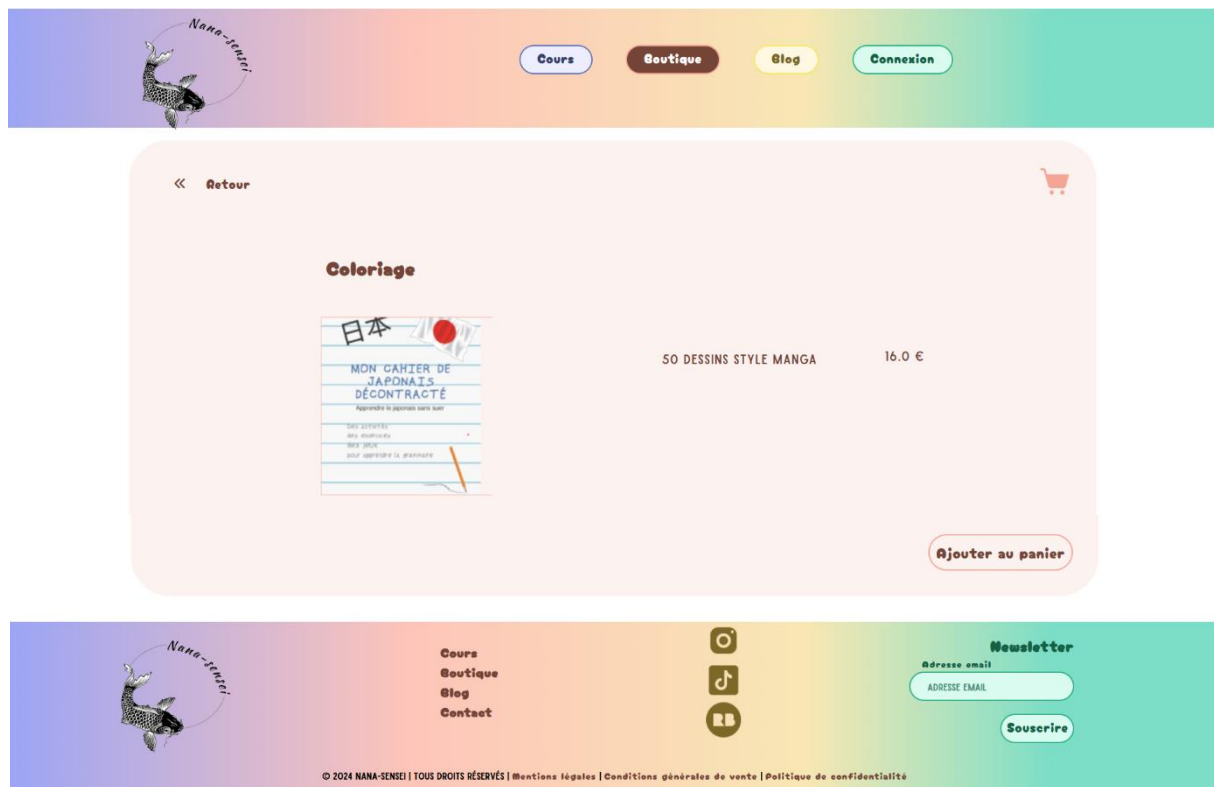
// Check if there are search results
if (searchResults != null) {
    productsToDisplay = searchResults;
    if (!searchResults.isEmpty()) {
        %<h2>Résultats de recherche pour "<%= searchTerm %>"</h2><%
    } else {
        %<p>Aucun résultat trouvé pour "<%= searchTerm %>"</p><%
    }
} else {
    // If no search results, display all products
    ProductDao productDao = new ProductDao(DbCon.getConnection());
    productsToDisplay = productDao.getAllProducts();
}

// Display products
if(!productsToDisplay.isEmpty()) {
    for(Product p : productsToDisplay) {>
        <div class="productcard"><a href="productboutique.jsp?id=<%=p.getId() %>">
            <div class="photocard">
                
            </div>
            <h2 class="nomproduit"> <%=p.getName() %></h2>
            <p class="descriptionproduit"> <%=p.getCategory() %></p>
            <p class="prix"><%=p.getPrice() %> €</p>
        </a></div>
        <% }
    } else {
        %<p>Aucun produit disponible.</p><%
    }
    %>
    </article>
</section>

    <%-- Include the footer.jsp file --%>
    <%@include file="includes/footer.jsp" %>
</body>
</html>

```


La page « Produit »



Cette page permet d’afficher de façon plus lisible toutes les informations des produits et suivant le choix soit l’ajouter au panier avec le bouton « ajouter au panier » qui utilise la servlet « addtocart », soit d’aller dans le panier via l’icône de caddy soit de retourner à la boutique via le lien situé sur le « retour ».

On retrouve :

- Les imports
- Les includes du head, du CSS commun aux pages de la boutique, le menu et le footer.
- Une requête SQL est préparée dès le début de la page car le titre de celle-ci s’adapte en fonction de l’article. Le reste de la page contient les informations récupérées de la base de données via cette requête préparée.

```

productboutique.jsp

<%-- Import necessary Java classes --%>
<%@page import="java.sql.PreparedStatement"%>
<%@page import="java.sql.ResultSet"%>
<%@page import="java.sql.Statement"%>
<%@page import="java.sql.Connection"%>
<%@page import="java.util.List"%>
<%@page import="com.nanasenseimvc.model.Product"%>
<%@page import="com.nanasenseimvc.connection.DbCon"%>
<%@page import="com.nanasenseimvc.dao.ProductDao"%>
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>

<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<%
// Establish database connection
Connection con = DbCon.getConnection();
// Get product ID from request parameter
String id = request.getParameter("id");
try {
    // Prepare SQL query to fetch product details
    String query = "SELECT * FROM produit WHERE idProduit=?";
    PreparedStatement pst = con.prepareStatement(query);
    pst.setString(1, id);
    // Execute query
    ResultSet rs = pst.executeQuery();

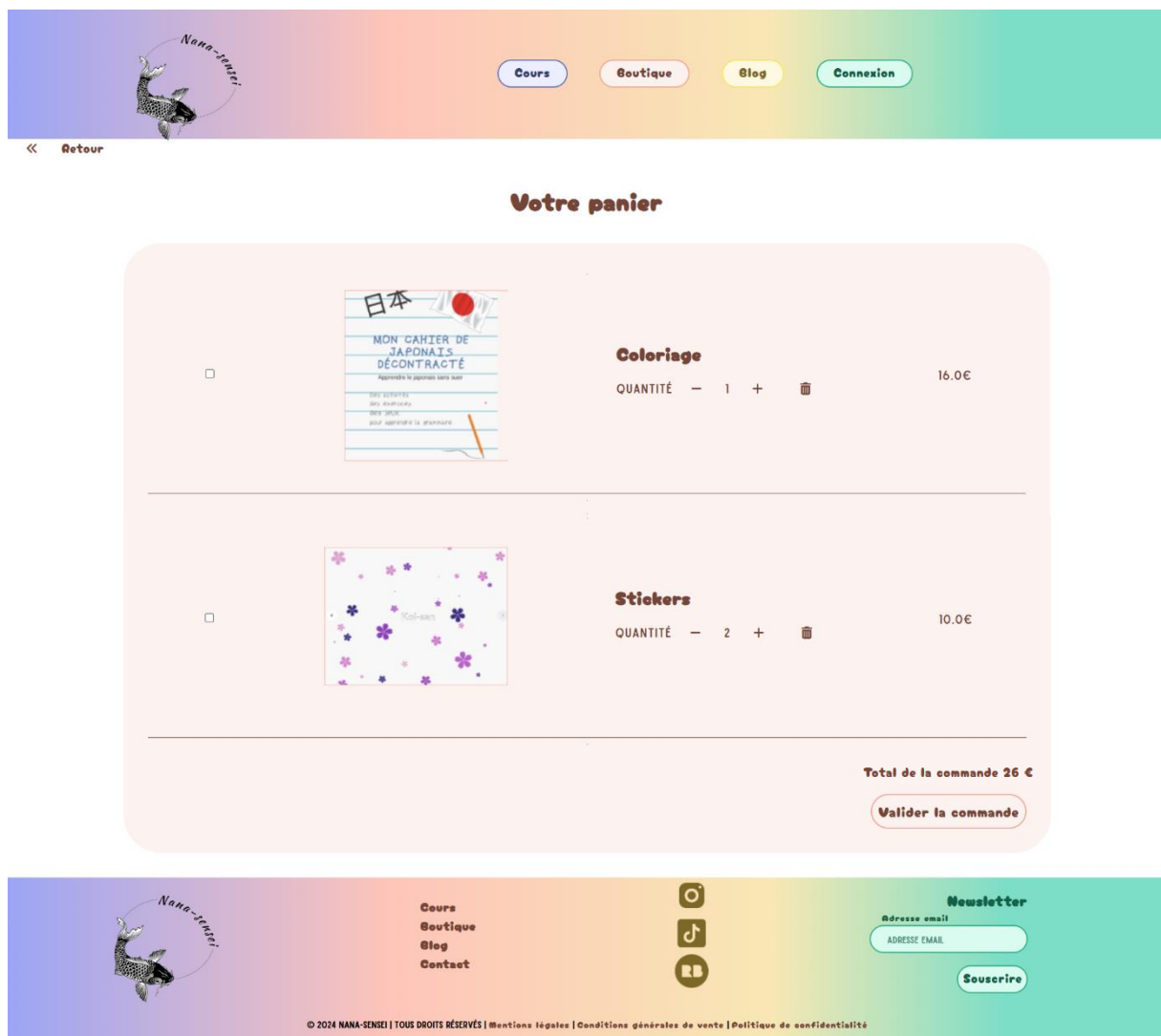
    // Process query results
    while (rs.next()) {
%>

<title><%=rs.getString(4) %></title>
<%-- Include head.jsp file --%>
<%@include file="includes/head.jsp" %>
<%-- Include CSS styles --%>
<style type="text/css">
<%@include file="CSS/boutique.css"%>
</style>
</head>
<body>
<%-- Include menu.jsp file --%>
<%@include file="includes/menu.jsp" %>
<section role="region" aria-labelledby="NomduProduit">
<div class="produit">
    <%-- Back to boutique link and cart icon --%>
    <div>
        <a href="boutique.jsp" class="back"><i class="fa-solid fa-angles-left"></i>
Retour </a>
        <a href="panier.jsp">
            
        </a>
    </div>
    <div class="product">
        <div class="titre">
            <%-- Hidden input for product ID --%>
            <input type="hidden" name="id" value="<%=rs.getString(1)%>">
            <h1 id="NomduProduit"> <%=rs.getString(4) %></h1>
            
        </div>
        <div>
            <p class="descriptionproduit"><%=rs.getString(5) %></p>
        </div>
        <div class="price">
            <p> <%=rs.getString(6) %> €</p><br>
        </div>
    </div>
    <%-- Add to cart button --%>
    <a class="addtocart" type="submit" href="addtocart?id=<%=rs.getString(1) %>"> Ajouter
au panier</a>
    </div>
    <%
    }
} catch (Exception e) {
    // Print error message if exception occurs
    System.out.println(e.getMessage());
}
%>

</section>
<%-- Include footer.jsp file --%>
<%@include file="includes/footer.jsp" %>
</body>
</html>

```

Le panier



Cette page permet de collecter les produits sélectionnés par l'utilisateur avant de passer sa commande.

On retrouve dans cette page :

- Les imports Java.
- Une partie Java qui prépare la page avec la déclaration du format du prix, la récupération du panier de la session, l'initialisation des variables. Si le panier n'est pas vide, une connexion à la base de données via le ProductDao et récupère les informations des produits grâce à la méthode « CartProducts() ». Pour finir on calcule le prix total de la commande avec la méthode « TotalCartPrice() ».
- Les includes du head, du CSS spécifique à la boutique, du menu et du footer.
- Un lien de retour à la boutique
- Les informations des produits que l'utilisateur a sélectionnés, des boutons pour augmenter ou diminuer la quantité des produits ce qui influence sur le prix total de l'article.
- Le prix total de tous les articles présents dans le panier et un bouton pour valider la commande.

```

panier.jsp

<%-- Import necessary Java classes --%>
<%@page import="java.text.DecimalFormat"%>
<%@page import="com.nanasenseimvc.connection.DbCon"%>
<%@page import="com.nanasenseimvc.dao.ProductDao"%>
<%@page import="java.util.List"%>
<%@page import="com.nanasenseimvc.model.Cart"%>
<%@page import="java.util.ArrayList"%>
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>

<%
//Create a DecimalFormat object with the pattern "#.##"
DecimalFormat dcf = new DecimalFormat("#.##");

//Set the DecimalFormat object as an attribute named "dcf" in the request
request.setAttribute("dcf", dcf);
// Retrieve the cart list from the session
ArrayList<Cart> cart_list = (ArrayList<Cart>) session.getAttribute("cart-list");

// Initialize variables for cart products and total
List<Cart> cartProduct = null;
double total = 0;

// If the cart list is not empty
if(cart_list != null) {
    // Get database connection
    ProductDao pDao = new ProductDao(DbCon.getConnection());
    // Retrieve cart products
    cartProduct = pDao.getCartProducts(cart_list);

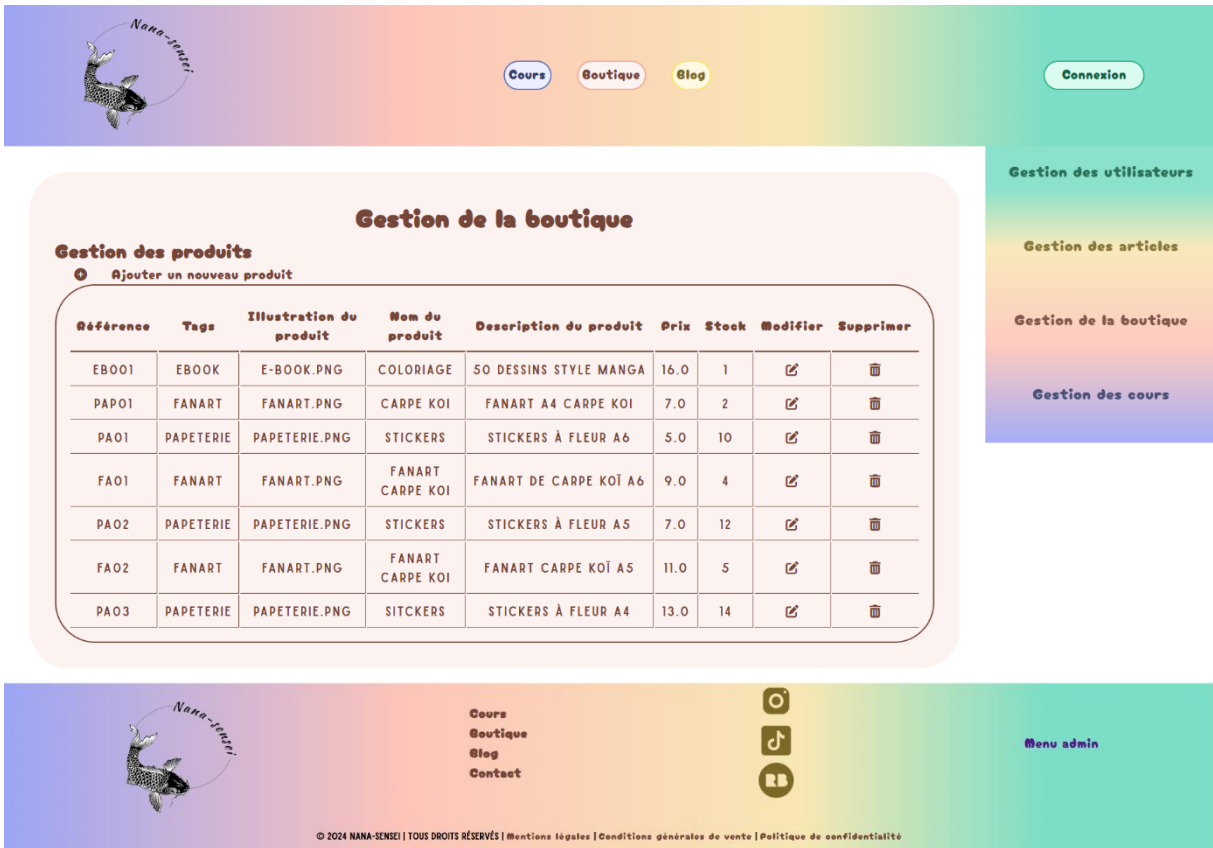
    // Calculate total price (commented out for now)
    total = pDao.getTotalCartPrice(cart_list);
    request.setAttribute("total", total);

    // Set cart list as a request attribute
    request.setAttribute("cart_list", cart_list);
}
%>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Panier | Nana-sensei </title>
    <%-- Include head.jsp file --%>
    <%@include file="includes/head.jsp" %>
    <%-- Include CSS styles --%>
    <style type="text/css">
    <%@include file="CSS/boutique.css"%>
    </style>
</head>
<body>
    <%-- Include menu.jsp file --%>
    <%@include file="includes/menu.jsp" %>
    <%-- Back to boutique Link --%>
    <div> <a href="boutique.jsp" class="Link"><i class="fa-solid fa-angles-left"></i> Retour
</a></div>
    <h1 id="panier">Votre panier</h1>

    <div class="produit">
    <%
// If cart is not empty, display cart items
if(cart_list != null) {
    for (Cart c : cartProduct) {
    %>
        <hr>
        <div class="cartproduct">
            <input type="hidden" name="id" value="<%=c.getId()%">
            <div><input type="checkbox"></div>
            <div> </div>
            <div>
                <div><h2 class="nomproduit"><%=c.getName() %"></h2></div>
                <div class="infoproduct">
                    <div><p class="price">Quantité</p></div>
                    <div class="price">
                        <%-- Decrease quantity link --%>
                        <a href="quantityindec?action=dec&id=<%=c.getId()%"><i class="fa-
solid fa-minus" ></i></a>
                        <%=c.getQuantity() %>
                        <%-- Increase quantity link --%>
                        <a href="quantityindec?action=inc&id=<%=c.getId()%"><i class="fa-
solid fa-plus" ></i></a>
                    </div>
                    <div class="price">
                        <%-- Remove item from cart link --%>
                        <a href="removeproductcart?id=<%=c.getId() %%" ><i class="fa-solid fa-
trash-can"></i></a>
                    </div>
                </div>
            </div>
            <div class="price"><%=c.getPrice() %></div>
        </div>
        <hr>
    <%
    }
    %>
    <!-- Paragraph to display the total order amount -->
    <p class="total">Total de la commande <%=total >0?dcf.format(total):0 %> €</p>
    <%-- Checkout button --%>
    <a class="addtocart" type="submit"> Valider la commande</a>
</div>
<!-- Include footer.jsp file -->
<%@include file="includes/footer.jsp" %>
</body>
</html>

```

La gestion de la boutique par l'administrateur



The screenshot shows the 'Gestion de la boutique' (Store Management) page for an administrator. The header features the 'Nana-Sensei' logo, navigation links for 'Cours', 'Boutique', and 'Blog', and a 'Connexion' button. A sidebar on the right lists management options: 'Gestion des utilisateurs', 'Gestion des articles', 'Gestion de la boutique' (highlighted), and 'Gestion des cours'. The main content area is titled 'Gestion des produits' and includes a link to 'Ajouter un nouveau produit'. Below this is a table listing products with columns for Reference, Tags, Illustration, Name, Description, Price, Stock, Modify, and Delete.

Référence	Tags	Illustration du produit	Nom du produit	Description du produit	Prix	Stock	Modifier	Supprimer
EBOOK	EBOOK	E-BOOK.PNG	COLORIAGE	50 DESSINS STYLE MANGA	16.0	1		
PAP01	FANART	FANART.PNG	CARPE KOI	FANART A4 CARPE KOI	7.0	2		
PA01	PAPETERIE	PAPETERIE.PNG	STICKERS	STICKERS À FLEUR A6	5.0	10		
FA01	FANART	FANART.PNG	FANART CARPE KOI	FANART DE CARPE KOI A6	9.0	4		
PA02	PAPETERIE	PAPETERIE.PNG	STICKERS	STICKERS À FLEUR A5	7.0	12		
FA02	FANART	FANART.PNG	FANART CARPE KOI	FANART CARPE KOI A5	11.0	5		
PA03	PAPETERIE	PAPETERIE.PNG	SITCKERS	STICKERS À FLEUR A4	13.0	14		

The footer contains the 'Nana-Sensei' logo, navigation links for 'Cours', 'Boutique', 'Blog', and 'Contact', social media icons, a 'Menu admin' link, and a copyright notice: '© 2024 NANA-SENSEI | TOUS DROITS RÉSERVÉS | Mentions légales | Conditions générales de vente | Politique de confidentialité'.

Cette page s'organise sensiblement de la même façon que les autres. On retrouve :

- Les imports Java.
- Les includes du head, du CSS spécifique à la boutique, du menu et du footer.
- Un menu de gestion lié à la connexion en tant qu'administrateur.
- Un lien d'ajout de produit.
- Une requête préparée se connecte à la base de données, récupère les informations de la table « produit » et affiche dans un tableau tous les produits créés tant qu'il y en a.
- Une icône qui mène à l'édition de produit grâce à un lien dans l'avant-dernière case du tableau.
- Une icône de poubelle qui est liée à la servlet « deleteproduct » qui sert à la suppression des produits dans la dernière case.

```

adminboutique.jsp

<%@page import="java.sql.PreparedStatement"%>
<%@page import="java.sql.ResultSet"%>
<%@page import="com.nanasenseimvc.connection.DbCon"%>
<%@page import="java.sql.Connection"%>
<% page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Gestion de la boutique | Nano-sensei</title>
<!-- Include common head content -->
<%@include file="../includes/head.jsp" %>
<!-- Include CSS styles for this page -->
<style>
    <%@include file="../CSS/adminboutique.css"%>
    <%@include file="../CSS/menuLog.css"%>
</style>
</head>
<body>
<!-- Include admin menu -->
<%@include file="adminmenu.jsp" %>
<main role="main">
<!-- Admin navigation menu -->
<div class="menuLog">
    <ul>
        <li class="account"><a href="adminUser.jsp" class="account"> Gestion des
utilisateurs</a></li>
        <li><a href="adminBlog.jsp" class="favoris"> Gestion des articles</a></li>
        <li class="commande"><a href="adminboutique.jsp" class="commande">Gestion de la
boutique</a></li>
        <li><a href="adminCours.jsp" class="mescours"> Gestion des cours </a></li>
    </ul>
</div>
<div class="container">
<h1>Gestion de la boutique</h1>

<h2>Gestion des produits </h2>
<!-- Button to add a new product -->
<div>
<a href="admincreaproduit.jsp" class="addproduct"><i class="fa-solid fa-circle-plus"></i>
Ajouter un nouveau produit</a>
</div>
<!-- Display status messages -->
<div class="valide">
<%
    String msg = request.getParameter("msg");
    if("done".equals(msg)){%>
        <h2 class="done">Modifications effectuées</h2>
    <%
    }if("wrong".equals(msg)){
    %>
        <h2 class="wrong">Il y a eu un problème. Merci de réessayer !</h2>
    <%> %>
    </div>
<!-- Table to display products -->
<table>
<thead>
<tr>
<th scope="col">Référence</th>
<th scope="col">Tags</th>
<th scope="col">Illustration du produit</th>
<th scope="col">Nom du produit</th>
<th scope="col">Description du produit</th>
<th scope="col">Prix</th>
<th scope="col">Stock</th>
<th scope="col">Modifier</th>
<th scope="col">Supprimer</th>
</tr>
</thead>
<tbody>
<%
    try{
        // Establish database connection
        Connection con= DbCon.getConnection();
        // SQL query to select all products
        String query = "SELECT * from produit";
        PreparedStatement pst = con.prepareStatement(query);
        ResultSet rs=pst.executeQuery();
        // Loop through each product in the result set
        while(rs.next()){
            %>
            <!-- Display product information in table rows -->
            <tr>
                <td scope="col"><%=rs.getString(2)%></td>
                <td scope="col"><%=rs.getString(8)%></td>
                <td scope="col"><%=rs.getString(3)%></td>
                <td scope="col"><%=rs.getString(4)%></td>
                <td scope="col" class="ellipse"><%=rs.getString(5)%></td>
                <td scope="col"><%=rs.getString(6)%></td>
                <td scope="col"><%=rs.getString(7)%></td>
                <td scope="col"><a href="editproduct.jsp?id=<%=rs.getString(1)%>"><i class="fa-solid fa-pen-
to-square"></i></a></td>
                <td scope="col"><a href=" ../deleproduct?id=<%=rs.getString(1)%>"><i class="fa-solid fa-
trash-can"></i></a></td>
            </tr>
            <%
        }
    } catch (Exception e){
        System.out.println(e.getMessage());
    }
    %>
</tbody>
</table>

</div>
</main>
<!-- Include footer -->
<%@include file="includes/footer.jsp" %>
</body>
</html>

```

La création de produit par l'administrateur

The screenshot shows the 'Ajouter un produit à la boutique' (Add product to the boutique) form in the Nana-Sensei admin interface. The form is titled 'Ajouter un produit à la boutique' and includes the following fields and elements:

- Product ID :** 8
- Photo de l'article:** A button labeled 'CHOISIR UN FICHIER' and a text input field labeled 'AUCUN FICHIER CHOISI'.
- Référence:** A text input field.
- Tags:** A dropdown menu with the selected tag '#E-BOOK'.
- Nom du produit:** A text input field.
- Description du produit:** A large text area for the product description.
- Prix:** A text input field.
- Stock:** A text input field.
- VALIDER:** A button to submit the form.

The interface also features a sidebar on the right with the following menu items:

- Gestion des utilisateurs
- Gestion des articles
- Gestion de la boutique
- Gestion des cours

The footer of the page includes the Nana-Sensei logo, a list of links (Cours, Boutique, Blog, Contact), social media icons (Instagram, YouTube, Facebook), a 'Menu admin' link, and a copyright notice: '© 2024 NANA-SENSEI | TOUS DROITS RÉSERVÉS | Mentions légales | Conditions générales de vente | Politique de confidentialité'.

Cette page sert à gérer la création des nouveaux produits de la boutique. Elle se compose :

- Des imports Java.
- Les includes du head, du CSS spécifique à la boutique, du menu et du footer.
- Une requête préparée qui se connecte à la base de données, récupère l'id maximal de la table « produit » et l'incrmente pour obtenir un nouvel id pour le nouveau produit.
- Un formulaire de création de produit avec tous les champs à remplir (image d'illustration, référence, tags, nom du produit, description du produit, prix et stock). Ce formulaire enregistre ces informations dans la base de données grâce à la servlet « addnewproduct »

```

admincreaproduct.jsp

<%@page import="java.sql.PreparedStatement"%>
<%@page import="java.sql.ResultSet"%>
<%@page import="com.nanasenseimvc.connection.DbCon"%>
<%@page import="java.sql.Connection"%>
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Nana-sensei | Création de produits</title>
<!-- Include common head content -->
<%@include file="../includes/head.jsp" %>
<!-- Include CSS styles for this page -->
<style>
    <%@include file="../CSS/adminboutique.css"%>
    <%@include file="../CSS/menuLog.css"%>
</style>
</head>
<body>
<!-- Include admin menu -->
<%@include file="adminmenu.jsp" %>
<main role="banner">
<!-- Admin navigation menu -->
    <div class="menuLog">
        <ul>
            <li><a href="adminUser.jsp" class="account"> Gestion des utilisateurs</a></li>
            <li><a href="adminBlog.jsp" class="favoris"> Gestion des articles</a></li>
            <li class="commande"><a href="adminboutique.jsp" class="commande">Gestion de la
boutique</a></li>
            <li><a href="adminCours.jsp" class="mescours"> Gestion des cours </a></li>
        </ul>
    </div>
    <div class="container">
<h1>Ajouter un produit à la boutique</h1>
<!-- Process of generating a new product ID -->
    <%
int id=1;
try {
    Connection con= DbCon.getConnection();
    String query = "SELECT max(idProduit) from produit;";
    PreparedStatement pst = con.prepareStatement(query);
    ResultSet rs=pst.executeQuery();
    while(rs.next()){
        id = rs.getInt(1);
        id++;
    }
} catch (Exception e){
    e.printStackTrace();
    System.out.println(e.getMessage());
}
%>
<!-- form for adding a new product -->
    <form action="../addnewproduct" method="post">
    <label>Product ID : <%out.println(id); %> </label>
    <input type="hidden" name="id" value="<%out.println(id); %>">
        <label>Photo de l'article</label>
        <input type="file" id="file" name="image">
        <label> Référence</label>
        <input type="text" name="reference">
        <label>Tags</label>
        <select name="tags">
            <option value="ebook">#e-book</option>
            <option value="fanart">#fanart</option>
            <option value="papeterie">#papeterie</option>
        </select>
        <label>Nom du produit</label>
        <input type="text" name="nom">
        <label>Description du produit</label>
        <textarea rows="5" name="description"></textarea>
        <div class="price">
        <label>Prix</label>
        <input class="price" type="number" name="prix">
        <label>Stock</label>
        <input class="price" type="number" name="stock"></div>
        <button type="submit">Valider</button>
    </form>
    </div>
</main>
<!-- Include footer -->
<%@include file="includes/footer.jsp" %>
</body>
</html>

```


L'édition de produit par l'administrateur

The screenshot shows the 'Modifier un produit' (Edit product) page in the Nana-Sensei admin interface. The page has a light pink background and is titled 'Modifier un produit'. It contains the following fields and elements:

- L'ID du produit :** 1
- Photo du produit:** A button labeled 'CHOISIR UN FICHIER' and a text input field containing 'AUCUN FICHIER CHOISI'.
- Référence:** A text input field containing 'EBOO1'.
- Tags:** A dropdown menu showing '#E-BOOK'.
- Nom du produit:** A text input field containing 'COLORIAGE'.
- Description du produit:** A large text area containing '50 DESSINS STYLE MANGA'.
- Prix:** A text input field containing '16,0'.
- Stock:** A text input field containing '1'.
- VALIDER:** A button at the bottom right of the form.

The interface also features a top navigation bar with links for 'Cours', 'Boutique', and 'Blog', and a user profile section for 'kyuzy'. A right sidebar contains links for 'Gestion des utilisateurs', 'Gestion des articles', 'Gestion de la boutique', and 'Gestion des cours'. The footer includes the Nana-Sensei logo, a list of links (Cours, Boutique, Blog, Contact), social media icons, a 'Menu admin' link, and a copyright notice: '© 2024 NANA-SENSEI | TOUS DROITS RÉSERVÉS | Mentions légales | Conditions générales de vente | Politique de confidentialité'.

Pour cette page d'édition de produit, elle se compose de la façon suivant :

- Les imports Java
- Les includes du head, du CSS spécifique à la boutique, du menu et du footer.
- Une requête préparée qui consiste à se connecter à la base de donnée, un produit est sélectionné via son ID et récupère divers champs de la base de données et les stocke dans des variables locales qui seront affichées dans le formulaire en valeur de base.
- Un formulaire contenant tous les champs d'un produit pour la modification grâce à la servlet « editproduct ».

```

editproduit.jsp

<%@page import="java.sql.PreparedStatement"%>
<%@page import="java.sql.ResultSet"%>
<%@page import="com.nanasensei.mvc.connection.DbCon"%>
<%@page import="java.sql.Connection"%>
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Edition de produit | Mana-sensei</title>
    <!-- Include common head content -->
    <%include file="../includes/head.jsp" %>
    <!-- Include CSS styles for this page -->
    <style>
        <%include file="../CSS/adminboutique.css"%>
        <%include file="../CSS/menuLog.css"%>
    </style>
</head>
<body>
    <!-- Include admin menu -->
    <%include file="adminmenu.jsp" %>
    <main role="banner">
        <!-- Admin navigation menu -->
        <div class="menuLog">
            <ul>
                <li><a href="adminUser.jsp" class="account"> Gestion des utilisateurs</a></li>
                <li><a href="adminBlog.jsp" class="favoris"> Gestion des articles</a></li>
                <li class="commande"><a href="adminboutique.jsp" class="commande">Gestion de la
boutique</a></li>
                <li><a href="adminCours.jsp" class="mescours"> Gestion des cours </a></li>
            </ul>
        </div>
        <div class="container">
            <h1>Modify a product in the shop</h1>
            <%
                // Get the product ID from the request parameter
                String id = request.getParameter("id");
                try {
                    // Establish database connection
                    Connection con = DbCon.getConnection();
                    // Prepare SQL query to fetch product details
                    String query = "SELECT * FROM produit WHERE idProduit = ?";
                    PreparedStatement pstmt = con.prepareStatement(query);
                    pstmt.setString(1, id);
                    ResultSet rs = pstmt.executeQuery();

                    // Iterate through the result set
                    while (rs.next()) {
                        // Retrieve product details from the result set
                        String reference = rs.getString("reference");
                        String tags = rs.getString("tags");
                        String nom = rs.getString("nomProduit");
                        String description = rs.getString("descriptionProduit");
                        double prix = rs.getDouble("prixUnitaireProduit");
                        int stock = rs.getInt("stock");

                        %>
                        <!-- Form to edit product details -->
                        <form action="../editproduct" method="post">
                            <label>L'ID du produit : <%= id %></label>
                            <input type="hidden" name="id" value="<%= id %>">
                            <label>Photo du produit</label>
                            <input type="file" id="file" name="image" accept=".jpg, .jpeg, .png, .svg,
image/jpeg, image/png, image/svg+xml">
                            <label>Référence</label>
                            <input type="text" name="reference" value="<%= reference %>">
                            <label>Tags</label>
                            <select name="tags">
                                <option value="ebook" <%= tags.equals("ebook") ? "selected" : "" %>>#e-
book</option>
                                <option value="fanart" <%= tags.equals("fanart") ? "selected" : ""
%>>#fanart</option>
                                <option value="papeterie" <%= tags.equals("papeterie") ? "selected" : ""
%>>#papeterie</option>
                            </select>
                            <label>Nom du produit</label>
                            <input type="text" name="nom" value="<%= nom %>">
                            <label>Description du produit</label>
                            <textarea rows="5" name="description"><%= description %></textarea>
                            <div class="price">
                                <label>Prix</label>
                                <input class="price" type="number" name="prix" value="<%= prix %>">
                                <label>Stock</label>
                                <input class="price" type="number" name="stock" value="<%= stock %>">
                            </div>
                            <button type="submit">Valider</button>
                        </form>
                        <%
                    }
                } catch (Exception e) {
                    // Print any exception message
                    System.out.println(e.getMessage());
                }
            %>
        </div>
    </main>
    <!-- Include footer -->
    <%include file="includes/footer.jsp" %>
</body>
</html>

```

Un fichier CSS commun à toute les pages liés à la boutique ...

```
h2{
  margin: 2rem;
  text-align: center;
  color: var(--blackboutique);
  font-family: 'Sailors';}

#boutique{
  margin: 2rem;
  text-align: center;
  color: var(--blackboutique);
  font-family: 'cherry bomb one';
  letter-spacing: 1px;}

/* barre de recherche*/
.search-container{
  width: 100%;
  display: flex;
  flex-direction: column;
  justify-content: center;
  margin: 1rem;}

.barresearch{
  display: flex;
  justify-content: center;}

form{
  width: 75%;
  display: flex;}

input#recherche{
  width: 60%;}

#recherche{
  border: 4px solid var(--flashboutique);
  border-radius: 50px;
  background-color: var(--lightboutique);
  margin-right: 2rem;}

::placeholder{
  padding: 1rem;
  color: var(--blackboutique);
  font-family: 'Sailors';
  opacity: 70%;}

.tag{
  width: 55%;
  display: flex;
  justify-content: center;
  align-items: center;
  margin-top: 1rem;}

#tags{
  width: fit-content;
  border: 2px solid var(--flashboutique);
  border-radius: 50px;
  background-color: var(--lightboutique);
  margin-right: 2rem;
  padding: 4px 8px;
  color: var(--blackboutique);
  font-family: 'Sailors';
  letter-spacing: 1px;}

.text{
  color: var(--blackboutique);
  font-family: 'cherry bomb one';
  letter-spacing: 1px;
  font-size: larger;
  margin-right: 1rem;}

.back{
  display: flex;
  justify-content: space-between;
  align-items: center;}

section .laboutique{
  background-color: var(--lightboutique);
  border-radius: 50px;
  margin: auto;
  width: 85%;
  display: flex;
  flex-wrap: wrap;}
```

```
/*Carte produit*/
article{
  width: 80%;
  margin: auto;
  display: flex;
  flex-wrap: wrap;
  justify-content: center;}

.productcard {
  margin: 1rem;
  max-width: 25%;
  border: 2px solid var(--flashboutique);
  border-radius: 50px;
  background-color: var(--lightboutique);
  padding: 2rem;
  color: var(--blackboutique);
  flex: 1 1 30%;
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;}

a{
  text-decoration: none;
  color: var(--blackboutique);
  display: flex;
  flex-direction: column;}

.photocard{
  width: 50%;
  align-self: center;
  margin-bottom: 1rem;}

.nomproduit{
  max-width: 30ch;
  font-family: 'cherry bomb one';
  letter-spacing: 1px;
  margin-bottom: 1rem;
  color: var(--blackboutique);}

.descriptionproduit{
  max-width: 30ch;
  font-family: 'Sailors';
  letter-spacing: 1px;
  margin-bottom: 1rem;}

.prix{
  text-align: right;
  font-family: 'Sailors';
  letter-spacing: 1px;}
```

```
/*Panier*/
#panier{
  margin: 2rem;
  text-align: center;
  color: var(--blackboutique);
  font-family: 'cherry bomb one';
  letter-spacing: 1px;}

.cartproduct{
  display: flex;
  align-items: center;
  justify-content: space-around;
  padding-bottom: 2rem;
  border-bottom: 1px solid var(--blackboutique);}

.infoproduct{
  display: flex;
  margin: 1rem 0rem;}

.total{
  text-align: right;
  color: var(--blackboutique);
  font-family: 'cherry bomb one';
  letter-spacing: 1px;
  margin: 1rem 0rem;}
```

```
/*page produit*/
.produit{
  width: 80%;
  margin: auto;
  padding: 2rem;
  background-color: var(--lightboutique);
  border-radius: 50px;
  margin-top: 1rem;
  margin-bottom: 1rem;
  display: flex;
  flex-direction: column;}

.Link{
  color: var(--blackboutique);
  font-family: 'cherry bomb one';
  letter-spacing: 1px;
  text-decoration: none;}

.titre{
  width: 40%;}

#NomduProduit{
  margin: 2rem;
  color: var(--blackboutique);
  font-family: 'cherry bomb one';
  letter-spacing: 1px;}

.product{
  width: 80%;
  margin: auto;
  margin-top: 2rem;
  margin-bottom: 2rem;
  display: flex;
  justify-content: space-evenly;
  align-items: center;
  color: var(--blackboutique);}

.descriptionproduit{
  max-width: 30ch;
  padding: 1rem;
  color: var(--blackboutique);}

.price{
  text-align: center;
  font-family: 'Sailors';
  letter-spacing: 1px;
  color: var(--blackboutique);}

.quantite{
  border: 2px solid var(--flashboutique);
  border-radius: 50px;
  background-color: var(--lightboutique);
  width: 100%;
  display: flex;
  flex-wrap: nowrap;
  align-items: center;
  color: var(--blackboutique);}

i{
  margin-left: 1.5em;
  margin-right: 1.5em;
  color: var(--blackboutique);}

.addtocart{
  border: 2px solid var(--flashboutique);
  border-radius: 50px;
  background-color: var(--lightboutique);
  padding: 8px;
  align-self: flex-end;
  font-family: 'cherry bomb one';
  letter-spacing: 1px;
  color: var(--blackboutique);
  font-size: large;}

.addtocart:hover {
  border-radius: 50px;
  background-color: var(--blackboutique);
  color: white;}
```

...avec ses media queries...

```
boutique.css

@media screen and (max-width: 800px){
  article{
    width:100%;
    margin: auto;
    display: flex;
    flex-direction: column;
    flex-wrap: wrap;
    justify-content: center;
  }
  .containerboutique{
    margin: 2rem ;
    background-color: var(--lightboutique);
    border-radius: 50px;
    padding: 2rem;
    display: flex;
    flex-direction: column;
  }
  #titreboutique {
    text-decoration: none;
    margin: 2rem;
    width: fit-content;
    border: solid;
    border-radius: 50px;
    border-color: var(--blackboutique);
    background-color: var(--pastelboutique);
    padding: 0.5rem 1rem 0.5rem 1rem;
    a {
      text-decoration: none;
      font-family: 'cherry Bomb One';
      letter-spacing: 1px;
      text-align: center;
      color: var(--blackboutique);
    }
  }
  .productcard {
    margin: auto;
    margin-top: 15px;
    max-width: 80%;
    border: 2px solid var(--flashboutique);
    border-radius: 50px;
    background-color: var(--lightboutique);
    padding: 1rem;
    color: var(--blackboutique);
    flex: 0 0 30%;
    min-width: 30%;
    display: flex;
    flex-direction: column;
    justify-content: center;
    a {
      text-decoration: none;
      text-align: center;
      color: var(--blackboutique);
      display: flex;
      flex-direction: column;
    }
  }
  .photocard{
    width: 50%;
    align-self: center;
    margin-bottom: 1rem;
  }
  .nomproduit{
    max-width: 30ch;
    font-family: 'cherry bomb one';
    letter-spacing: 1px;
    margin-bottom: 1rem;
  }
  .descriptionproduit{
    max-width: 30ch;
    font-family: 'Sailors';
    letter-spacing: 1px;
    margin-bottom: 1rem;
  }
  .prix{
    text-align: right;
    font-family: 'Sailors';
    letter-spacing: 1px;
  }
}
```

```
adminboutique.css

.container{
  background-color: var(--lightboutique);
  border-radius: 50px;
  margin: 2rem;
  padding: 2rem;
  width: 80%;
}
li.commande{
  border: 3px solid var(--flashboutique);
}
i{
  margin-left: 1.5em;
  margin-right: 1.5em;
  color: var(--blackboutique);
}
h1{
  font-family: 'cherry bomb one';
  letter-spacing: 1px;
  color: var(--blackboutique);
  text-align: center;
}
h2{
  font-family: 'cherry bomb one';
  letter-spacing: 1px;
  color: var(--blackboutique);
}
form{
  display: flex;
  flex-direction: column;
  justify-content: center;
  width: 65%;
  margin: auto;
}
label{
  font-family: 'cherry bomb one';
  letter-spacing: 1px;
  color: var(--blackboutique);
  margin-bottom: 0.75rem;
}
input:not(#file, .price), select, option, textarea{
  border: 1px solid var(--flashboutique);
  border-radius: 25px;
  padding: 0.5rem 0.75rem;
  width: 65%;
  font-family: 'Sailors' !important;
  color: var(--blackboutique);
  margin-bottom: 1.25rem;
}
.addproduct{
  text-decoration: none;
  color: var(--blackboutique);
  font-family: 'cherry Bomb One';
  letter-spacing: 1px;
}
#file {
  border: 1px solid var(--flashboutique);
  border-radius: 50px;
  width: 65%;
  font-family: 'Sailors';
  color: var(--blackboutique);
  margin-bottom: 1.25rem;
}
::file-selector-button{
  border: 2px solid var(--flashboutique);
  border-radius: 50px;
  background-color: var(--lightboutique);
  padding: 1rem;
  width: fit-content;
  font-family: 'Sailors';
  color: var(--blackboutique);
}
button{
  border: 2px solid var(--flashboutique);
  border-radius: 50px;
  background-color: var(--lightboutique);
  padding: 0.25rem;
  width: fit-content;
  font-family: 'Sailors' !important;
  color: var(--blackboutique);
  align-self: center;
}
.price{
  display: flex;
  width: 65%;
  justify-content: space-around;
  align-items: baseline;
  input{
    width:10ch;
    border: 1px solid var(--flashboutique);
    border-radius: 50px;
    padding: 0.5rem 0.75rem;
    font-family: 'Sailors';
    color: var(--blackboutique);
    margin-bottom: 1.25rem;
  }
}
.titre{
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin: 1rem;
}
```

... et son CSS dédié à la partie administrateur.

```
adminboutique.css

table{
  border-radius: 50px;
  border: 2px solid var(--blackboutique);
  padding: 1rem;
}
th{
  padding: 0.5rem;
  border-bottom: 2px solid var(--blackboutique);
  font-family: 'Cherry Bomb One';
  color: var(--blackboutique);
  letter-spacing: 2px;
  margin-bottom: 2rem;
  margin-right: 1rem;
}
td{
  padding: 0.5rem;
  border-bottom: 1px solid var(--blackboutique);
  font-family: 'Sailors';
  color: var(--blackboutique);
  text-align: center;
  letter-spacing: 2px;
  margin-bottom: 2rem;
  margin-right: 1rem;
}
td:not(:last-child) {
  border-right: 1px solid var(--blackboutique);
}
.elipse{
  text-overflow: ellipsis;
  overflow: hidden;
  white-space: nowrap;
}
.valide .done{
  text-align: center;
  color: green;
}
.valide .wrong{
  text-align: center;
  color: red;
}
```

Les contrôleurs

La servlet qui permet de rechercher un produit dans la boutique

```
package com.nanasenseimvc.controller;

// Import necessary Java and Servlet libraries
import java.io.IOException;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

// Import custom classes
import com.nanasenseimvc.connection.DbCon;
import com.nanasenseimvc.model.Product;

// Define the servlet mapping URL
@WebServlet("/searchproduct")
public class SearchProductServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    // Constructor
    public SearchProductServlet() {
        super();
    }

    // Handle POST requests
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        // Set response content type
        response.setContentType("text/html; charset=UTF-8");

        // Use try-with-resources to automatically close the PrintWriter
        try (PrintWriter out = response.getWriter()) {
            // Get the search term from the request parameters
            String search = request.getParameter("search");

            // Create a list to store search results
            List<Product> searchResults = new ArrayList<Product>();

            // SQL query to search for products
            String query = "SELECT * FROM produit WHERE nomProduit LIKE ? OR descriptionProduit LIKE ?";

            // Establish database connection and execute query
            Connection con = DbCon.getConnection();
            PreparedStatement pst = con.prepareStatement(query);

            // Set search parameters for the query
            pst.setString(1, "%" + search + "%");
            pst.setString(2, "%" + search + "%");

            // Execute the query and process results
            ResultSet rs = pst.executeQuery();
            while (rs.next()) {
                // Create a new Product object for each result
                Product row = new Product();
                row.setId(rs.getInt("idproduit"));
                row.setImage(rs.getString("photoProduit"));
                row.setName(rs.getString("nomProduit"));
                row.setCategory(rs.getString("descriptionProduit"));
                row.setPrice(rs.getDouble("prixUnitaireProduit"));
                row.setStock(rs.getInt("stock"));
                row.setTags(rs.getString("tags"));

                // Add the product to the search results list
                searchResults.add(row);
            }

            // Set attributes for the JSP page
            request.setAttribute("searchResults", searchResults);
            request.setAttribute("searchTerm", search);

            // Forward the request to the boutique.jsp page
            request.getRequestDispatcher("boutique.jsp").forward(request, response);
        } catch (SQLException | ClassNotFoundException e) {
            // Log any errors that occur during the search process
            getServletContext().log("Erreur lors de la recherche de produits", e);

            // Redirect to the boutique page with an error parameter
            response.sendRedirect("boutique.jsp?error=search_error");
            return;
        }
    }
}
```

Cette servlet sert à réaliser une recherche sur la page d'accueil de la boutique. Elle fonctionne de la manière suivante :

- On récupère le contenu de l'input de name="search" dans le formulaire où la servlet est appelée.
- On crée une liste pour stocker les résultats
- On rédige une requête SQL qui sélectionnera les produits quand le nom du produit ou sa description ressembleront à ce qu'on lui donnera.
- On établit la connexion avec la base de données.
- On demande d'exécuter la requête préparée en amont en lui indiquant qu'elle doit sélectionner les produits par rapport à l'input de name="search".
- Tant qu'il y a des résultats, la requête sera exécutée.
- On crée un nouvel objet Product pour chaque résultat puis on l'ajoute dans la liste de résultat de recherche.
- On définit les attributs que l'on utilisera dans la page JSP pour gérer les résultats qui apparaîtront dans la page boutique.jsp.

La servlet qui permet d'ajouter un produit dans le panier

Cette servlet permet d'ajouter un produit choisi par l'utilisateur dans le panier. Elle fonctionne de la façon suivante :

- On obtient l'ID du produit à partir des paramètres de la requête.
- On crée un nouvel élément de panier avec l'ID donné et la quantité 1.
- On obtient la session en cours et on récupère la liste de panier existante à partir de la session.
- Si le panier n'existe pas, en créer un nouveau et ajouter le produit.
- On vérifie si le produit existe déjà dans le panier et s'il existe, on augmente la quantité.
- Si le produit n'existe pas dans le panier, l'ajouter.
- Mettre à jour la liste de panier dans la session et on redirige vers la page du panier.

```
package com.nanasenseimvc.controller;

import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

import com.nanasenseimvc.model.Cart;

// Servlet mapping for "/addtocart" URL
@WebServlet("/addtocart")
public class AddToCartServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    public AddToCartServlet() {
        super();
    }

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        response.setContentType("text/html; charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            // Get the product ID from the request parameters
            int id = Integer.parseInt(request.getParameter("id"));

            // Create a new Cart item with the given ID and quantity 1
            Cart cm = new Cart();
            cm.setId(id);
            cm.setQuantity(1);

            // Get the current session
            HttpSession session = request.getSession();

            // Retrieve the existing cart list from the session
            ArrayList<Cart> cartList = (ArrayList<Cart>) session.getAttribute("cart-list");

            if (cartList == null) {
                // If the cart doesn't exist, create a new one and add the item
                cartList = new ArrayList<>();
                cartList.add(cm);
            }
        }
    }
}
```

```
    } else {
        boolean exists = false;
        // Check if the product already exists in the cart
        for (Cart c : cartList) {
            if (c.getId() == id) {
                // If it exists, increase the quantity
                c.setQuantity(c.getQuantity() + 1);
                exists = true;
                break; // Exit the loop once the product is found
            }
        }
        if (!exists) {
            // If the product doesn't exist in the cart, add it
            cartList.add(cm);
        }
    }
    // Update the cart list in the session
    session.setAttribute("cart-list", cartList);
    // Redirect to the cart page
    response.sendRedirect("panier.jsp");
} catch (NumberFormatException e) {
    // Handle the error if the ID is not a valid number
    // For example, display an error message to the user
}
}
```

La servlet qui permet de retirer un produit du panier

Cette servlet permet de retirer un produit du panier. Elle fonctionne de la manière suivante :

- On obtient l'ID du produit à partir des paramètres de la requête.
- On récupère la liste du panier à partir de la session.
- On recherche le produit à supprimer du panier et on le supprime le produit s'il est trouvé.
- On met à jour la liste du panier dans la session et on redirige l'utilisateur vers la page du panier.

```
RemoveProductCartServlet.java

package com.nanasenseimvc.controller;

import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.nanasenseimvc.model.Cart;

// Servlet mapping for "/removeproductcart" URL
@WebServlet("/removeproductcart")
public class RemoveProductCartServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    public RemoveProductCartServlet() {
        super();
    }

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
    ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            // Get the product ID from the request parameters
            String productId = request.getParameter("id");
            if (productId != null && !productId.isEmpty()) {
                // Retrieve the cart list from the session
                ArrayList<Cart> cart_list = (ArrayList<Cart>)
                request.getSession().getAttribute("cart-list");
                if (cart_list != null) {
                    // Convert the product ID to an integer
                    int id = Integer.parseInt(productId);
                    // Find the item to remove from the cart
                    Cart itemToRemove = null;
                    for (Cart c : cart_list) {
                        if (c.getId() == id) {
                            itemToRemove = c;
                            break;
                        }
                    }
                    // Remove the item if found
                    if (itemToRemove != null) {
                        cart_list.remove(itemToRemove);
                        // Update the cart list in the session
                        request.getSession().setAttribute("cart-list", cart_list);
                    }
                }
            }
            // Redirect to the cart page
            response.sendRedirect("panier.jsp");
        } catch (NumberFormatException e) {
            // Handle number format exception (invalid product ID)
            e.printStackTrace();
            response.sendRedirect("error.jsp");
        } catch (Exception e) {
            // Handle any other exceptions
            e.printStackTrace();
            response.sendRedirect("error.jsp");
        }
    }
}
```

La servlet qui permet de changer la quantité d'un produit

Cette servlet permet à l'utilisateur d'augmenter ou de diminuer la quantité des produits dans le panier avant de valider la commande. Elle fonctionne de la manière suivante :

- On obtient l'action (augmentation ou diminution) et l'ID du produit à partir des paramètres de la requête.
- On récupère le panier à partir de la session.
- On vérifie si l'action, l'ID et le panier sont valides.
- On parcourt les produits du panier.
- Si l'action est "inc", on augmente la quantité.
- Si l'action est "dec" et que la quantité est supérieure à 1, on diminue la quantité.
- On met à jour la quantité d'articles et on enregistre les modifications dans la session.
- On redirige ensuite l'utilisateur vers le panier avec la bonne quantité indiquée.

```
package com.nanasenseimvc.controller;

import java.io.IOException;
import java.util.ArrayList;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.nanasenseimvc.model.Cart;

// This servlet handles requests to increase or decrease the quantity of items in the cart
@WebServlet("/quantityindec")
public class QuantityIncDecServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    public QuantityIncDecServlet() {
        super();
    }

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
        ServletException, IOException {
        // Set the response content type
        response.setContentType("text/html; charset=UTF-8");

        try {
            // Get the action (increase or decrease) and item ID from the request parameters
            String action = request.getParameter("action");
            int id = Integer.parseInt(request.getParameter("id"));

            // Retrieve the cart list from the session
            ArrayList<Cart> cart_list = (ArrayList<Cart>)
            request.getSession().getAttribute("cart-list");

            // Check if action, id, and cart_list are valid
            if (action != null && id >= 1 && cart_list != null) {
                boolean updated = false;

                // Iterate through the cart items
                for (Cart c : cart_list) {
                    if (c.getId() == id) {
                        int quantity = c.getQuantity();

                        // Increase quantity if action is "inc"
                        if ("inc".equals(action)) {
                            quantity++;
                            updated = true;
                        }
                    }
                }
            }
        }
    }
}
```

```
QuantityIncDecServlet.java

// Decrease quantity if action is "dec" and quantity is greater than 1
else if ("dec".equals(action) && quantity > 1) {
    quantity--;
    updated = true;
}

// Update the item quantity
c.setQuantity(quantity);
break; // Exit the loop after finding the item
}

// If the cart was updated, save the changes to the session
if (updated) {
    request.getSession().setAttribute("cart-list", cart_list);
}
} catch (NumberFormatException e) {
    // Handle exception if the ID is not a valid number
    e.printStackTrace();
} catch (Exception e) {
    // Handle other potential exceptions
    e.printStackTrace();
}

// Redirect the user back to the cart page
response.sendRedirect("panier.jsp");
}
```


La servlet qui permet de créer un produit

Ce servlet gère les demandes d'ajout d'un nouveau produit à la base de données. Elle fonctionne de la manière suivante :

- Elle récupère les détails du produit à partir des paramètres de la demande.
- Elle crée une instance ProductDao et établit une connexion à la base de données.
- Elle ajoute le nouveau produit à la base de données grâce à la méthode « addProduct() » défini dans le ProductDao.
- Si le produit est ajouté à la base de donnée, elle redirige l'administrateur vers la page de la boutique d'administration avec un message de réussite
- S'il y a eu une erreur et que le produit n'est pas ajouté à la base de données, elle redirige l'administrateur vers la page d'ajout d'un nouveau produit avec un message d'erreur

```
package com.nanasenseimvc.controller;

import java.io.IOException;
import java.io.PrintWriter;
import java.sql.SQLException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.nanasenseimvc.connection.DbCon;
import com.nanasenseimvc.dao.ProductDao;
import com.nanasenseimvc.model.Product;

// This servlet handles requests to add a new product to the database
@WebServlet("/addnewproduct")
public class AddNewProductServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    public AddNewProductServlet() {
        super();
    }

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
        // Set the response content type
        response.setContentType("text/html ; charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            // Retrieve product details from request parameters
            String id = request.getParameter("id");
            String reference = request.getParameter("reference");
            String image = request.getParameter("image");
            String name = request.getParameter("nom");
            String category = request.getParameter("description");
            double price = Double.parseDouble(request.getParameter("prix"));
            int stock = Integer.parseInt(request.getParameter("stock"));
            String tags = request.getParameter("tags");
            // Create a ProductDao instance and establish a database connection
            ProductDao pdao = new ProductDao(DbCon.getConnection());
            // Add the new product to the database
            Product product = pdao.addProduct(reference, image, name, category, price, 0,
tags);
            // Log success message
            System.out.println("Product added");
            // Redirect to the admin boutique page with a success message
            response.sendRedirect("admin/adminboutique.jsp?msg=done");
        } catch (ClassNotFoundException | SQLException e) {
            // Log error message if product addition fails
            System.out.println("No product added");
            // Redirect to the add new product page with an error message
            response.sendRedirect("addNewProduct.jsp?msg=wrong");
        }
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
        // TODO Auto-generated method stub
        doGet(request, response);
    }
}
```

La servlet qui permet de modifier un produit

Cette servlet permet de gérer la modification des détails d'un produit dans une boutique en ligne. Elle fonctionne de la manière suivante :

- Elle récupère les détails du produit à partir des paramètres de requête.
- Une requête préparée SQL met à jour les détails du produit dans la base de données à l'aide d'espaces réservés pour les paramètres.
- Elle établit une connexion à la base de données.
- Elle crée une instruction préparée pour exécuter en toute sécurité la requête SQL avec des entrées paramétrées.
- Elle définit les valeurs de chaque espace réservé dans la requête SQL.
- Elle exécute la requête de mise à jour et obtient le nombre de lignes affectés.
- Si au moins une ligne est mise à jour, elle redirige vers la page d'administration avec un message de réussite.
- Sinon elle redirige vers la page d'administration avec un message d'erreur en cas d'échec.



```
package com.nanasenseimvc.controller;

// Import necessary Java and Servlet libraries
import java.io.IOException;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.PreparedStatement;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.nanasenseimvc.connection.DbCon;

// Servlet mapping for the "/editproduct" URL
@WebServlet("/editproduct")
public class EditProductServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    // Constructor
    public EditProductServlet() {
        super();
    }

    // Handle GET requests
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        // Set response content type and character encoding to handle UTF-8 characters
        response.setContentType("text/html ; charset=UTF-8");
        request.setCharacterEncoding("UTF-8");

        // Retrieve product details from HTTP request parameters
        String id = request.getParameter("id"); // Product ID
        String reference = request.getParameter("reference"); // Product reference
        String image = request.getParameter("image"); // Product image URL or path
        String name = request.getParameter("nom"); // Product name
        String category = request.getParameter("description"); // Product description or category
        double price = Double.parseDouble(request.getParameter("prix")); // Product price (converted to double)
        int stock = Integer.parseInt(request.getParameter("stock")); // Product stock quantity (converted to integer)
        String tags = request.getParameter("tags"); // Tags associated with the product

        try (PrintWriter out = response.getWriter()) {
            // SQL query to update product details in the database using placeholders for parameters
            String query = "UPDATE produit SET reference=?, nomProduit=?, descriptionProduit=?, " +
                "prixUnitaireProduit=?, photoProduit=?, stock=?, tags=? WHERE idProduit=?";

            // Establish database connection using a utility class (DbCon)
            Connection con = DbCon.getConnection();

            // Create a PreparedStatement to safely execute the SQL query with parameterized inputs
            PreparedStatement pstmt = con.prepareStatement(query);

            // Set the values for each placeholder in the SQL query
            pstmt.setString(1, reference); // Set reference value
            pstmt.setString(2, name); // Set product name value
            pstmt.setString(3, category); // Set description/category value
            pstmt.setDouble(4, price); // Set price value
            pstmt.setString(5, image); // Set image URL/path value
            pstmt.setInt(6, stock); // Set stock quantity value
            pstmt.setString(7, tags); // Set tags value
            pstmt.setString(8, id); // Set product ID value

            // Execute the update query and get the number of affected rows
            int rowsAffected = pstmt.executeUpdate();

            // If at least one row is updated, redirect to admin page with success message
            if (rowsAffected > 0) {
                response.sendRedirect("admin/adminboutique.jsp?msg=done");
            }

            // Log error message if the update fails for any reason
            catch (Exception e) {
                System.out.println("The product " + id + " " + reference + " " + name + " didn't update");
                // Redirect to admin page with an error message in case of failure
                response.sendRedirect("admin/adminboutique.jsp?msg=wrong");
            }

            // Handle POST requests by delegating them to the doGet method for processing
            protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
                doGet(request, response);
            }
        }
    }
}
```

La servlet qui permet de supprimer un produit

Cette servlet permet la gestion des demandes de suppression de produits. Elle fonctionne de la manière suivante :

- Elle obtient l’ID du produit à partir des paramètres de la demande et vérifie qu’il n’est pas nul.
- Elle crée une instance de ProductDao avec une connexion à la base de données.
- Elle utilise la méthode présente dans ProductDao « deleteProduct () » pour supprimer le produit identifié grâce à son ID.
- Si la suppression est réussie, elle redirige vers la page de gestion de la boutique.
- Si la suppression a échoué, elle redirige vers la page de gestion de la boutique.

```
DeleteProductServlet.java

package com.nanasenseimvc.controller;

import java.io.IOException;
import java.io.PrintWriter;
import java.sql.SQLException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.nanasenseimvc.connection.DbCon;
import com.nanasenseimvc.dao.ProductDao;

// Servlet mapping for "/deleteproduct" URL
@WebServlet("/deleteproduct")
public class DeleteProductServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    // Constructor
    public DeleteProductServlet() {
        super();
    }
    // Handle GET requests
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
        // Set response content type
        response.setContentType("text/html ; charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            // Get the product ID from the request parameters
            String id = request.getParameter("id");
            // Check if ID is not null
            if (id != null) {
                // Create ProductDao instance with database connection
                ProductDao productDao = new ProductDao(DbCon.getConnection());
                // Delete the product using the provided ID
                productDao.deleteProduct(Integer.parseInt(id));
                // Log successful deletion
                System.out.println("Product deleted");
                // Redirect to admin page with success message
                response.sendRedirect("admin/adminboutique.jsp?msg=done");
            } else {
                // Log failed deletion due to null ID
                System.out.println("Product didn't delete");
                // Redirect to admin page with error message
                response.sendRedirect("admin/adminboutique.jsp?msg=wrong");
            }
        } catch (SQLException | ClassNotFoundException e) {
            // Print stack trace and error message for exceptions
            e.printStackTrace();
            System.out.println(e.getMessage());
        }
    }
}
```

La base de donnée

1 • SELECT * FROM nana_sensei.produit;

	idProduit	reference	photoProduit	nomProduit	descriptionProduit	prixUnitaireProduit	Stock	tags
1	eb01	e-book.png		Coloriage	50 dessins style manga	16	1	ebook
3	pa01	papeterie.png		Stickers	stickers à fleur A6	5	10	papeterie
4	fa01	fanart.png		Fanart Carpe...	Fanart de carpe koi A6	9	4	fanart
5	pa02	papeterie.png		stickers	stickers à fleur A5	7	12	papeterie
6	fa02	fanart.png		Fanart carpe koi	Fanart carpe koi A5	11	5	fanart
7	pa03	papeterie.png		Stickers	Stickers à fleur A4	13	14	papeterie
10	pa04	logo.png		Logo Nana-se...	Stickers logo nana-sensei A5	5	15	papeterie
11	pa05	logo.png		Logo Nana-se...	Sticker logo nana-sensei a6	4	10	papeterie

La table « Produit » de la base de données qui est utilisé dans la boutique et dans la gestion de la boutique par l'administrateur.

3. Le déploiement

Pour faciliter son déploiement et sa gestion, nous avons choisi d'intégrer ce projet à Docker. Bien qu'il ne soit pas opérationnel, il me semblait important de le mentionner car cette démarche entre dans le cadre de bonnes pratiques de développement.

La configuration Docker comprend un Dockerfile pour construire l'image de l'application ainsi qu'un fichier docker-compose.yml pour gérer le déploiement de l'application et de sa base de données MySQL.

docker-compose.yml

```
version: "2.4"
services:
  mysqladb:
    container_name: mysqlcontainerfornanasensei
    image: mysql:8.0
    restart: unless-stopped
    command: --default-authentication-plugin=mysql_native_password
    environment:
      - MYSQL_ROOT_PASSWORD=@Irfa2024!
      - MYSQL_DATABASE=nana_sensei
      - SERVICE_TAGS=prod
      - SERVICE_NAME=mysqladb
    ports:
      - 3306:3306
    expose:
      - 3307
    volumes:
      - $HOME/database:/var/lib/mysql
    networks:
      - apinetwork
  dev-projetnanasenseimvc:
    build: .
    image: tomcat-eb:v.9.0.17
    container_name: nanasenseimvc-project-container
    volumes:
      - ./webapp:/usr/local/tomcat/webapps
    command: catalina.sh run
    ports:
      - 8081:8080
    restart: unless-stopped
    depends_on:
      - mysqladb
    networks:
      - apinetwork
networks:
  apinetwork:
    driver: bridge
```

Dockerfile

```
FROM tomcat:9-jre11
LABEL Author="Cathy NAVARRO"
EXPOSE 8081
COPY target/* /usr/local/tomcat/webapps/
COPY target/nanasenseimvc.war /usr/local/tomcat/webapps/ROOT.war

CMD ["catalina.sh", "run"]
```

```
PS C:\Users\stagiaire\eclipse-workspace\nanasenseimvc> docker-compose up -d
time="2025-01-23T15:46:08+01:00" level=warning msg="The \"HOME\" variable is not set. Defaulting to a blank string."
time="2025-01-23T15:46:08+01:00" level=warning msg="C:\Users\stagiaire\eclipse-workspace\nanasenseimvc\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 3/3
 ✓ Network nanasenseimvc_apinetwork          Created           0.0s
 ✓ Container mysqlcontainerfornanasensei     Started             0.8s
 ✓ Container nanasenseimvc-project-container Started             1.0s
```

```
PS C:\Users\stagiaire\eclipse-workspace\nanasenseimvc> docker-compose up --build -d
time="2025-01-23T15:59:43+01:00" level=warning msg="The \"HOME\" variable is not set. Defaulting to a blank string."
time="2025-01-23T15:59:43+01:00" level=warning msg="C:\Users\stagiaire\eclipse-workspace\nanasenseimvc\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 0/0
[+] Running 0/1projctnanasenseimvc Building                                0.1s
[+] Building 1.4s (9/9) FINISHED                                           docker:desktop-linux
=> [dev-projetnanasenseimvc internal] load build definition from Dockerfile
=> => transferring dockerfile: 241B                                         0.0s
=> [dev-projetnanasenseimvc internal] load metadata for docker.io/library/tomcat:9-jre11 1.0s
=> [dev-projetnanasenseimvc internal] load .dockerignore                                                            0.0s
=> => transferring context: 2B                                              0.0s
=> [dev-projetnanasenseimvc 1/3] FROM docker.io/library/tomcat:9-jre11@sha256:ffb8fef4e3211c1897d928b00e77d91d5ef47c3f85036580aecee0533db903e0 0.0s
=> => resolve docker.io/library/tomcat:9-jre11@sha256:ffb8fef4e3211c1897d928b00e77d91d5ef47c3f85036580aecee0533db903e0 0.0s
=> [dev-projetnanasenseimvc internal] load build context                                                            0.1s
=> => transferring context: 11.17kB                                         0.0s
=> CACHED [dev-projetnanasenseimvc 2/3] COPY target/* /usr/local/tomcat/webapps/ 0.0s
=> CACHED [dev-projetnanasenseimvc 3/3] COPY target/nanasenseimvc.war /usr/local/tomcat/webapps/ROOT.war 0.0s
=> [dev-projetnanasenseimvc] exporting to image                                                                    0.1s
=> => exporting layers                                                       0.0s
=> => exporting manifest sha256:99deefcafd310535efbe9b5a6f327530ae327518eeb976cb4c014444b06099 0.0s
=> => exporting config sha256:98c8d0106573239ab872871c7f0bf0252227b439ed00ddf5c4c0b3f2b7956d3c 0.0s
=> => exporting attestation manifest sha256:b41aac1d731add447c2d9d8f5296c1c4f49a90d90e5ad888a4be54a21e8114b0 0.0s
=> => exporting manifest list sha256:f486a3fe564d3014ffb6566ce5292a2c8b9aec40fa42a109b713e2501a39ebbd 0.0s
=> => naming to docker.io/library/tomcat-eb:v.9.0.17                     0.0s
=> => unpacking to docker.io/library/tomcat-eb:v.9.0.17                  0.0s
=> [dev-projetnanasenseimvc] resolving provenance for metadata file                                              0.0s
[+] Running 3/33T15:59:43+01:00" level=warning msg="current commit information was not captured by the build" error="git was not found in the system: exec: \"git.exe"
 ✓ Service dev-projetnanasenseimvc Built 1.5s
 ✓ Container mysqlcontainerfornanasensei Running 0.0s
 ✓ Container nanasenseimvc-project-container Started 1.3s
```

4. Les limites et voies d'amélioration du projet

a) Limites du projet

Ce site suivant mon apprentissage du développement web, j'ai relevé plusieurs défis en apportant des solutions viables et une méthodologie de réflexion qui me permettront de m'épanouir dans ce nouveau domaine de compétence.

- La découverte et l'appréhension du responsive qui m'a demandé beaucoup d'entraînement (principalement sur Frontend Mentor).
- La gestion des erreurs grâce auxquels j'ai pu améliorer ma compréhension de l'anglais technique puisqu'il s'agit de la langue de la plupart des forums.
- Utiliser et m'appropriier les différents langages de programmation en utilisant les documentations des ceux-ci.
- Actuellement un problème avec Docker qui ne m'affiche pas les pages de mon projet ni ma base de données.

b) Voies d'amélioration du projet

Dans l'état actuel du projet, plusieurs points ne sont pas suffisant pour que le site soit utilisable :

- La sécurité avec l'ajout de token CSRF pour sécuriser la relation application web, navigateur et utilisateurs.
- La quantité d'information stockée, principalement pour les utilisateurs qui pourraient avoir besoin de plusieurs adresses.
- L'optimisation des performances avec la minification du CSS et le format des images

c) Avenir du projet

Ce projet est prévu pour un déploiement début du deuxième semestre de 2025. Plusieurs fonctionnalités supplémentaires sont prévues entre aujourd'hui et fin 2025 :

- Pour la section des cours :
 - Mise en place de réservation de cours en ligne
 - Mise en place de supports de cours téléchargeables
- Pour la section de la boutique :
 - **Finalisation du paiement en ligne (PRIORITÉ)**
 - Mise en place d'un programme de fidélité
- Pour la section du blog :
 - Création du blog
 - Mise en place d'un système de modération des commentaires

En plus de ces évolutions prévues, un changement de langage de programmation est aussi envisagé pour faciliter le déploiement et la maintenance et passer de JSP à une application sous Angular et Symfony qui avec mes connaissances actuelles me paraissent plus adaptés au projet.

III. Conclusion

Ce projet m'a permis de mettre en application les différents outils et langages de programmation vu lors de la formation. J'ai acquis une méthodologie de projet efficace ainsi qu'une méthodologie d'apprentissage des langages grâce auxquelles j'ai pu mener mes stages à bien en toute autonomie.

Consciente des progrès qu'il me reste à faire, le domaine du développement évolue rapidement et une veille technologique via des newsletters ou des sites internet est indispensable.

Je remercie mes 2 professeurs qui ont su partager leurs connaissances efficacement ainsi que mes collègues de formation avec qui la cohésion a été d'un soutien important.

"L'homme se découvre quand il se mesure avec l'obstacle"

Antoine de Saint-Exupéry "Terre des Hommes", 1939

Par la suite, j'ai déjà des projets de développement en cours, comme celui d'une plateforme qui me permettra par la suite de trouver plus facilement des clients pour l'entreprise de création de site web et application que je souhaite ouvrir.